AI LAB

Contents

UNINFORMED SEARCH 1
INFORMED SEARCH: 19
MAZE BFS 26
Maze Multi-Goal Best-First Search 37
GREEDY BEST FIRST SEARCH: 40
CODE DIFFERENCES: 43
A* Search 46
EXAMPLE: 55
DYNAMIC A* SEARCGH: 59
INFORMED SEARCH CONCLUSION 68
LOCAL SEARCHES 71
HILL CLIMBING 86
N QUEENS: 90
8 PUZZLE 100
GENETIC ALGORITHM: 111
SVM 127
PANDAS 142
KMEANS 162
MM & HMM 167

UNINFORMED SEARCH

BFS

```python
# Depth First Search

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': [],
    'F': ['I'],
    'G': [],
    'H': [],
    'I': []
}

def dfs(graph, start, goal):

    visited = []
    stack = []

    visited.append(start)
    stack.append(start)

    while stack:

        node = stack.pop()
        print(node, end=" ")

        if node == goal:
            print("\nGoal Found")
            return

        for neighbour in reversed(graph[node]):
            if neighbour not in visited:
                visited.append(neighbour)
                stack.append(neighbour)
```

```python

start = 'A'
goal = 'I'

print("DFS Traversal:")
dfs(graph, start, goal)
```

Line by Line Explanation

python

```
tree = {
    'A': ['B', 'C'],   # A has children B and C
    'B': ['D', 'E'],   # B has children D and E
    'C': ['F', 'G'],   # C has children F and G
    'D': ['H'],        # D has child H
    'E': [],           # E has no children (leaf node)
    'F': ['I'],        # F has child I
    'G': [],           # G has no children (leaf node)
    'H': [],           # H has no children (leaf node)
    'I': []            # I has no children — this is our GOAL
}
```

---

python

```
def bfs(graph, start, goal):

    visited = []   # stores nodes we have already seen
                   # so we never visit the same node twice

    queue = []     # the waiting line — nodes to be explored
                   # BFS uses a queue (First In First Out)
                   # meaning whatever entered first leaves first

    visited.append(start)   # mark start node A as visited
    queue.append(start)     # put start node A in the queue
                            # queue = ['A']
                            # visited = ['A']
```

python

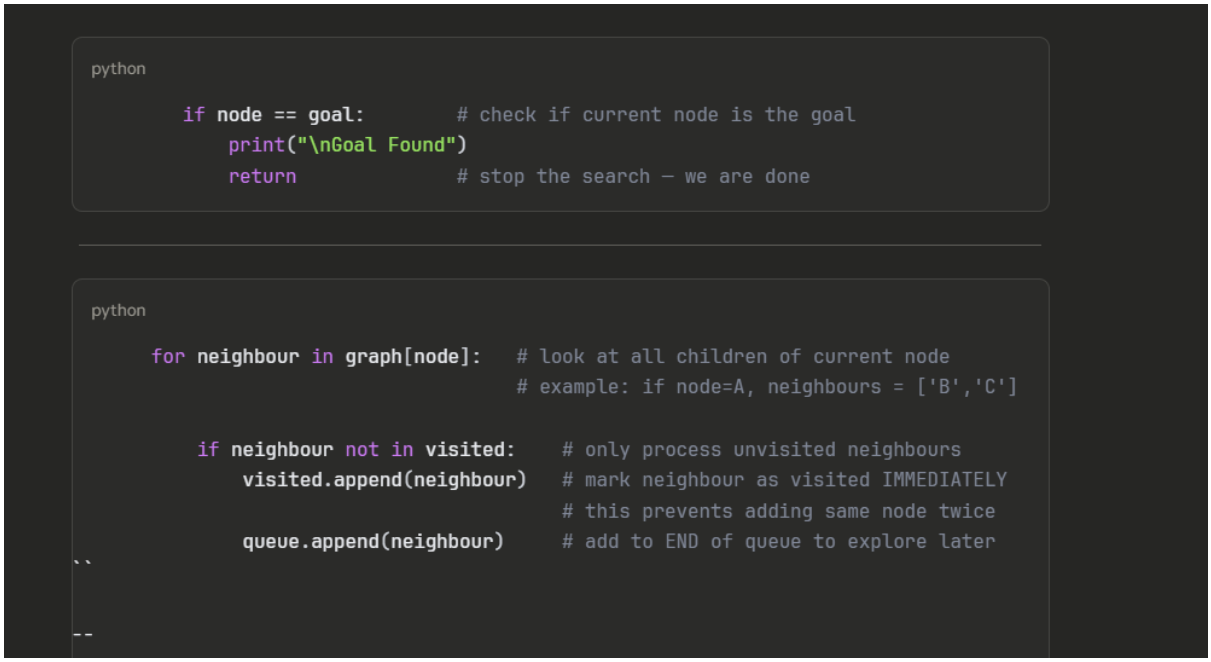
```
    while queue:   # keep running as long as queue is not empty
```

This is the **main loop.** Every time we visit a node, we remove it from queue. Loop stops when queue becomes empty or goal is found.

---

python

```
        node = queue.pop(0)   # remove and get the FIRST node from queue
                              # pop(0) = take from FRONT (this is what makes it BFS)
                              # if we used pop() from back it would be DFS
        print(node, end=" ")  # print the node we are currently visiting
```



```python
        if node == goal:        # check if current node is the goal
            print("\nGoal Found")
            return              # stop the search — we are done
```

```python
      for neighbour in graph[node]:   # look at all children of current node
                                      # example: if node=A, neighbours = ['B','C']

          if neighbour not in visited:    # only process unvisited neighbours
              visited.append(neighbour)   # mark neighbour as visited IMMEDIATELY
                                          # this prevents adding same node twice
              queue.append(neighbour)     # add to END of queue to explore later
```

--

DFS

```python
# Depth First Search

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': [],
    'F': ['I'],
    'G': [],
    'H': [],
    'I': []
}

def dfs(graph, start, goal):

    visited = []
    stack = []

    visited.append(start)
    stack.append(start)

    while stack:

        node = stack.pop()
        print(node, end=" ")

        if node == goal:
            print("\nGoal Found")
            return

        for neighbour in reversed(graph[node]):
            if neighbour not in visited:
                visited.append(neighbour)
                stack.append(neighbour)
```

```python
                    stack.append(neighbour)

start = 'A'
goal = 'I'

print("DFS Traversal:")
dfs(graph, start, goal)
```

```python
graph = {
    'A': ['B', 'C'],   # A's neighbours are B and C
    'B': ['D', 'E'],   # B's neighbours are D and E
    'C': ['F', 'G'],   # C's neighbours are F and G
    'D': ['H'],        # D's neighbour is H
    'E': [],           # E is a leaf — no neighbours
    'F': ['I'],        # F's neighbour is I (GOAL)
    'G': [],           # G is a leaf — no neighbours
    'H': [],           # H is a leaf — no neighbours
    'I': []            # I is the GOAL — no neighbours
}
```

---

python

```python
    visited = []          # tracks nodes already seen
    stack   = []          # the waiting list — but works as a STACK
                          # Stack = Last In First Out (LIFO)
                          # meaning whatever entered LAST leaves FIRST
                          # THIS is the only real difference from BFS
                          # BFS used queue (pop from front)
                          # DFS uses stack (pop from back)

    visited.append(start)  # mark A as ted  → visited = ['A']
    stack.append(start)    # push A onto rack  → stack   = ['A']
```

python

```python
    while stack:           # keep running while stack is not empty

        node = stack.pop() # pop() removes from the BACK (last item)
                           # this is STACK behaviour — Last In First Out
                           # BFS used pop(0) — front
                           # DFS uses pop()  — back  ← KEY DIFFERENCE

        print(node, end=" ")  # print current node being visited

        if node == goal:      # check if goal reached
            print("\nGoal Found")
            return
```

```python
        for neighbour in reversed(graph[node]):
```

This line needs special attention. `reversed()` flips the neighbour list.

Why? Because we are using a stack. If neighbours of A are `['B', 'C']` and we push them in order, C goes in last so C comes out first. But we want B first. So we **reverse before pushing** so that B ends up on top of stack.

```python
# Without reversed:  push B then C  → stack top is C → visits C first ❌
# With reversed:     push C then B  → stack top is B → visits B first ✅
```

```python
            if neighbour not in visited:   # only process unvisited neighbours
                visited.append(neighbour)  # mark as visited immediately
                stack.append(neighbour)    # push onto stack
```

↓

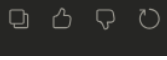
```python
            if neighbour not in visited:   # only process unvisited neighbours
                visited.append(neighbour)  # mark as visited immediately
                stack.append(neighbour)    # push onto stack
```


BFS vs DFS — The ONE difference

```python
# BFS — uses QUEUE
node = queue.pop(0)   # takes from FRONT → level by level

# DFS — uses STACK
node = stack.pop()    # takes from BACK → goes deep first
```

How each explores the tree

```
BFS (level by level):     A → B → C → D → E → F → G → H → I
                          explores all of level 1 before level 2

DFS (deep first):         A → B → D → H → E → C → F → I
                          goes as deep as possible before backtracking
```



DLS

Code Blame 34 lines (27 loc) · 575 Bytes

```python
# Depth Limited Search

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': [],
    'F': ['I'],
    'G': [],
    'H': [],
    'I': []
}

def dls(node, goal, depth):

    if depth == 0 and node == goal:
        print("Goal Found:", node)
        return True

    if depth > 0:
        for child in graph[node]:
            if dls(child, goal, depth - 1):
                print(node, end=" ")
                return True

    return False

start = 'A'
goal = 'I'
depth_limit = 3

print("DLS Search:")
dls(start, goal, depth_limit)
```

The Tree Structure with Depth Levels

```
Depth 0 →        A
               /   \
Depth 1 →     B     C
             / \   / \
Depth 2 →   D   E F   G
            |     |
Depth 3 →   H     I  ← GOAL
```

`depth_limit = 3` means we are **allowed to go maximum 3 levels deep** from start.

Key Concept — Recursion

Unlike BFS and DFS which used loops, DLS uses **recursion** — meaning the function **calls itself** with a smaller depth each time.

```python
dls(node, goal, depth)  # calls itself as
dls(child, goal, depth - 1)  # depth reduces by 1 each time
```

Line by Line Explanation

```python
def dls(node, goal, depth):
```

Three parameters:

- `node` = the current node we are checking
- `goal` = the node we are looking for (`'I'`)
- `depth` = how many more levels we are allowed to go deeper

```python
    if depth == 0 and node == goal:
        print("Goal Found:", node)
        return True
```

This is the **success condition** — two things must be true at the same time:

```python
    if depth > 0:
        for child in graph[node]:
```

If we still have depth remaining, look at all children of current node. `graph[node]` gives the list of children — same as BFS and DFS.

```python
            if dls(child, goal, depth - 1):
```

This is the **recursive call** — call the same function on the child but with `depth - 1`. This means:

- We go one level deeper
- But we have one less level of depth remaining

```python
                print(node, end=" ")
                return True
```

This is important — we print the node **AFTER** the recursive call returns True. This is why the output prints **backwards** — `I` first then `F` then `C` then `A`. It prints while unwinding back up the tree.

```python
    return False
```

If depth is 0 and node is not goal, OR all children were explored and goal not found

---

Step by Step Trace — What happens internally

```
dls('A', 'I', 3)          depth=3, A≠goal, check children B and C
│
├── dls('B', 'I', 2)       depth=2, B≠goal, check children D and E
│   ├── dls('D', 'I', 1)   depth=1, D≠goal, check child H
│   │   └── dls('H','I',0) depth=0, H≠goal → return False
│   │
│   └── dls('E', 'I', 1)   depth=1, E≠goal, no children
│       → return False
│   → return False (B side found nothing)
│
└── dls('C', 'I', 2)       depth=2, C≠goal, check children F and G
    ├── dls('F', 'I', 1)   depth=1, F≠goal, check child I
    │   └── dls('I','I',0) depth=0, I==goal → print "Goal Found: I"
    │                                       → return True ✅
    │   ← True received → print 'F' → return True
    ← True received → print 'C' → return True
← True received → print 'A' → return True
```

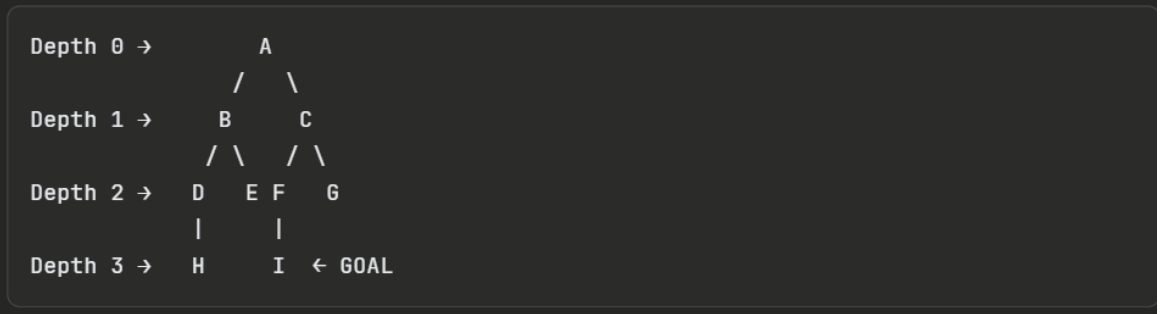
Why output is printed backwards

```
Goal Found: I    ← printed first  (deepest point)
F                ← printed second (F's call returned True)
C                ← printed third  (C's call returned True)
A                ← printed last   (A's call returned True)
```

The path is actually `A → C → F → I` but printed in reverse because `print(node)` ru

IDDFS:

The Tree Structure



Key Concept — What is Iterative Deepening?

IDDFS = runs DLS **multiple times**, increasing depth limit by 1 each time until goal is found.

```
Try depth 0 → only check A          → not found
Try depth 1 → check A, B, C         → not found
Try depth 2 → check A,B,C,D,E,F,G → not found
Try depth 3 → reaches H and I       → FOUND ✅
```

It combines the **memory efficiency of DFS** with the **completeness of BFS**.

This code has TWO functions — let me explain both

Function 1 — `dls` (same as before but with `path` added)

```python
def dls(node, goal, depth, path):
```

Same 3 parameters as before PLUS one new one:

- `path` = a list that **collects the route** taken to reach the goal

```python
    if depth == 0 and node == goal:
        path.append(node)    # add GOAL node to path first
        return True          # goal found — tell parent call
```

When goal is found, add it to path and return True. Path starts building here from the deepest point.

When goal is found, add it to path and return True. Path starts building here from the deepest point.

```python
    if depth > 0:
        for child in tree[node]:
            if dls(child, goal, depth-1, path):  # recursive call
                path.append(node)   # child found goal, so add ME to path
                return True         # tell MY parent i found it too
    return False
```

If a child found the goal, add current node to path and bubble True upward. This builds the path **backwards** as it unwinds — just like previous DLS.

Function 2 — `iterative_deepening`

```python
def iterative_deepening(start, goal, max_depth):
```

- `start` = starting node `'A'`
- `goal` = target node `'I'`
- `max_depth` = maximum depth to try = `5`

```python
    for depth in range(max_depth + 1):
```

`range(5 + 1)` = `range(6)` = `0, 1, 2, 3, 4, 5`

This loop tries **depth 0, then 1, then 2, then 3...** one by one.

```python
        path = []
```

Reset path to empty list at the start of **every depth attempt.** We don't want old failed attempts mixing with new ones.

```python
        if dls(start, goal, depth, path):
            print("Path:", " -> ".join(reversed(path)))
            return
```

Call DLS with current depth limit. If goal found:

- `path` at this point = `['I', 'F', 'C', 'A']` (backwards)
- `reversed(path)` = `['A', 'C', 'F', 'I']` (correct order)
- `" -> ".join(...)` joins them with arrows = `"A -> C -> F -> I"`

````
    print("Goal Not Found")
```

If all depths tried and goal never found, print this.

---

## Full Trace — What happens at each depth

**Depth = 0:** Only checks if A == I → No
```
dls('A', 'I', 0) → depth=0, A≠I → False
```

**Depth = 1:** Checks A and its immediate children
```
dls('A','I',1) → checks B(depth0) and C(depth0)
  B≠I, C≠I → False
```

**Depth = 2:** Goes 2 levels deep
```
dls('A','I',2) → checks B(depth1), C(depth1)
  B → checks D(depth0), E(depth0) → D≠I, E≠I
  C → checks F(depth0), G(depth0) → F≠I, G≠I → False
````

Comparison: DLS vs IDDFS

	DLS	IDDFS
Depth limit	Fixed — you set one limit	Tries ALL limits from 0 to max
If limit too small	Returns False, gives up	Automatically tries next depth
Finds path?	Only if limit is correct	Always finds if goal exists
Path stored?	No (just prints nodes)	Yes (uses `path` list)

The big idea: IDDFS is just DLS run in a loop with increasing depth — simple but very powerful!

UCS

Line by Line Explanation

```python
graph = {
    'A': {'B': 2, 'C': 1},   # A connects to B(cost 2) and C(cost 1)
    'B': {'D': 4, 'E': 3},   # B connects to D(cost 4) and E(cost 3)
    'C': {'F': 1, 'G': 5},   # C connects to F(cost 1) and G(cost 5)
    'D': {'H': 2},            # D connects to H(cost 2)
    'E': {},                  # E is leaf — no neighbours
    'F': {'I': 6},            # F connects to I(cost 6) — I is GOAL
    'G': {},                  # G is leaf
    'H': {},                  # H is leaf
    'I': {}                   # I is GOAL — no outgoing edges
}
```

```python
    frontier = [(start, 0)]
```

Frontier stores `(node, total_cost_to_reach_node)`. We start at A with cost 0.

```
frontier = [('A', 0)]
```

```python
    visited = set()
```

Tracks nodes already explored. Uses `set()` this time (not list like BFS/DFS) — set is faster for checking membership.

```python
    while frontier:
        frontier.sort(key=lambda x: x[1])
```

Sort frontier by `x[1]` which is the **cost** (second element of each tuple). UCS always picks the **cheapest cumulative cost** node first.

```python
        node, cost = frontier.pop(0)
```

Pop the first item after sorting — this is the cheapest node.

- `node` = the node name e.g. `'A'`
- `cost` = total cost to reach this node from start e.g. `0`

```python
        if node in visited:
            continue
```

Skip if already visited. Same node can appear multiple times in frontier with different costs — we only want to process it once with the cheapest cost.

```python
        visited.add(node)
```

Mark as visited now that we are processing it.

Mark as visited now that we are processing it.

```python
        if node == goal:
            path = []
            while node:
                path.append(node)
                node = parent[node]
            path.reverse()
            print("Path:", path)
            print("Total Cost:", cost)
            return
```

Goal found — reconstruct path by backtracking through `parent`.

Note: `while node:` — this loop runs until `node` becomes `None`. Since `parent[start] = None`, when we reach start and do `node = parent['A']` it becomes `None` and loop stops.

```python
        for neighbour, weight in graph[node].items():
```

`.items()` gives us both the neighbour name AND the edge cost together.

- `neighbour` = connected node name
- `weight` = cost of that edge

Frontier changes every iteration

```python
frontier = [('A', 0)]    # starts with only A
```

Iteration 1 — node = A

```python
frontier.sort(...)           # [('A', 0)]
node, cost = frontier.pop(0) # node='A', cost=0
                             # frontier = []  ← now empty

# then we expand A's neighbours:
for neighbour, weight in graph['A'].items():
    # graph['A'] = {'B': 2, 'C': 1}
    # neighbour='B', weight=2 → frontier.append(('B', 0+2))
    # neighbour='C', weight=1 → frontier.append(('C', 0+1))

# frontier = [('B', 2), ('C', 1)]  ← now has 2 nodes
```

```python
            if neighbour not in visited:
                parent[neighbour] = node
                frontier.append((neighbour, cost + weight))
```

- Save the parent of neighbour
- Add neighbour to frontier with **total cost = current cost + edge weight**
- Example: at A(cost=0), going to C(weight=1) → C added as `('C', 0+1=1)`

The problem — we only know the GOAL at the end

When UCS reaches `I`, it knows the goal is found. But it does NOT have the full path stored anywhere. It only has `parent` which tells us **one step back** at a time.

So to get the full path we have to **backtrack step by step**:

```python
path = []
while node:                  # node starts as 'I'
    path.append(node)        # add current node to path
    node = parent[node]      # move ONE step back to parent
```

What happens each time `node = parent[node]` runs

```
Start:  node = 'I'
path.append('I')     → path = ['I']
node = parent['I']   → node = 'F'     ← moved back one step

path.append('F')     → path = ['I', 'F']
node = parent['F']   → node = 'C'     ← moved back one step

path.append('C')     → path = ['I', 'F', 'C']
node = parent['C']   → node = 'A'     ← moved back one step

path.append('A')     → path = ['I', 'F', 'C', 'A']
node = parent['A']   → node = None    ← A has no parent

while node → while None → LOOP STOPS
```

```
## Simple analogy

Think of `parent` like a chain of breadcrumbs:
```

```
I → F → C → A → None
```

`node = parent[node]` just means **"follow the breadcrumb back one step"** each time until you reach the start where there are no more breadcrumbs (`None`).

Without this line, you would be stuck at `I` forever and never know how you got there.

INFORMED SEARCH:

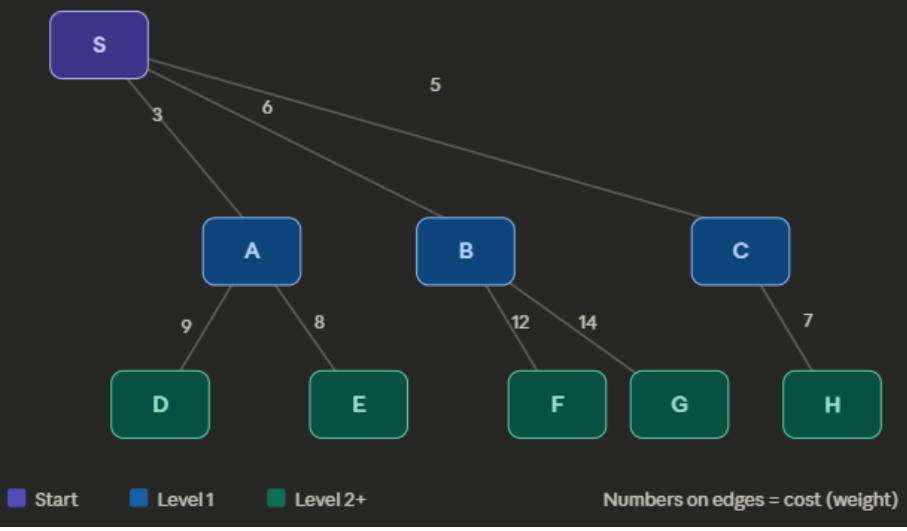
Best First Search — Explained Simply

What's the Core Idea?

Normal search explores neighbors blindly. Best First Search is **greedy** — at every step, it picks the neighbor with the **lowest edge cost** and goes there first.

It uses a **Priority Queue** (min-heap) instead of a regular queue, so the cheapest option always comes out first.

The Graph (Visual)



How the Code Works — Step by Step

The Priority Queue (PQ) is the heart of the algorithm. Think of it as a line where everyone has a ticket number — the lowest number always goes first.

```
pq.put((0, 'S'))   →  PQ: [(0, S)]
```

The `visited` set makes sure we never process the same node twice.

Walking through `best_first_search(graph, 'A', 'I')`:

The search starts at `A`. Here's the PQ state at each step:

Step	Node Popped	PQ After	Visited
1	`A` (cost 0)	[(8,E), (9,D)]	{A}
2	`E` (cost 8)	[(9,D)]	{A, E} — leaf, no children
3	`D` (cost 9)	[]	{A, E, D} — leaf, no children

Goal `I` is never reached from `A` — so the search exhausts all reachable nodes and exits.

Important Limitation to Know

Best First Search is **greedy** — it picks the cheapest *individual edge*, not the cheapest *total path*. This means it can go down a path that looks cheap at first but turns out to be very long overall. It's fast, but **not guaranteed to find the shortest total path** (that's what Dijkstra's or A* are for).

---

python

```python
graph = {
    'S': [('A',3), ('B',6), ('C',5)],
    'A': [('D',9), ('E',8)],
    ...
}
```

Defines the graph as an **adjacency list** (dictionary).

- Each key is a **node**
- Each value is a list of **(neighbor, heuristic_weight)** tuples
- The weight represents how "promising" that neighbor is — **lower = better**

Example: `'S': [('A',3), ('B',6), ('C',5)]` means S connects to A (cost 3), B (cost 6), C (cost 5)

Function Definition & Setup

python

```python
def best_first_search(graph, start, goal):
    visited = set()
```

- Defines the function taking the graph, start node, and goal node
- `visited = set()` — tracks already-explored nodes so we **never revisit** them

`set()` in Python means:

🔹 **An empty set**

Python Run

```python
visited = set()
```

👉 This creates an **empty collection** that will store unique values.

🔹 **What is a set?**

A **set** is a data structure that:

1. ✅ Stores **unique elements only** (no duplicates)
2. ⚡ Is **very fast** for checking:

   Python Run

   ```python
   if x in visited
   ```

3. ❌ Does NOT keep order

```python
pq = PriorityQueue()
pq.put((0, start))
```

- Creates an empty priority queue
- Inserts the **start node with cost 0** — the journey begins here with no cost yet

---

🔹 Why use `set()` here (in your code)?

In Best-First Search:

```python
if node not in visited:
```

This check is **very fast with a set** (faster than list).

🔹 Set vs List (important ⚡)

Feature	set()	list []
Duplicates	❌ Not allowed	✅ Allowed
Search speed	⚡ Fast	🐢 Slower
Order	❌ No order	✅ Ordered

Main Loop

```python
while not pq.empty():
    cost, node = pq.get()
```

- Keeps running as long as there are nodes to explore
- `pq.get()` pops the node with the **lowest cost** (most promising)

Example: If PQ has `[(3,A), (6,B), (5,C)]`, it pops `(3, A)` first

Get Plus ×

What does this line do?

```python
cost, node = pq.get()
```

👉 It does **two things at once:**

1. Gets the smallest element

Example:

```python
(3, 'A')
```

2. Unpacks it into two variables

- `cost = 3`
- `node = 'A'`

Think of it like this

```python
temp = pq.get()   # temp = (3, 'A')
cost = temp[0]    # 3
node = temp[1]    # 'A'
```

Run

But Python lets you write it shorter:

```python
cost, node = pq.get()
```

Run

🔹 Think of it like this

```python
temp = pq.get()   # temp = (3, 'A')
cost = temp[0]    # 3
node = temp[1]    # 'A'
```

But Python lets you write it shorter:

```python
cost, node = pq.get()
```

🔹 Why is it important?

Because Best-First Search always:

> 🔍 Picks the node with the **lowest heuristic value (cost)** first

```python
print(node, end=" ")
```

👉 Prints the value of `node` **on the same line**, followed by a space.

🔹 Without `end=" "`:

Each print goes to a new line.

🔹 With `end=" "`:

Output looks like:

```
A B C D
```

✔ Used here to show traversal in one line.

Example: If PQ has `[(3,A), (6,B), (5,C)]`, it pops `(3, A)` first

```python
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
```

- Skips the node if already visited (avoids duplicate processing)
- Otherwise, **prints it** (shows exploration order) and marks it visited

```python
            if node == goal:
                print("\nGoal Reached")
                return
```

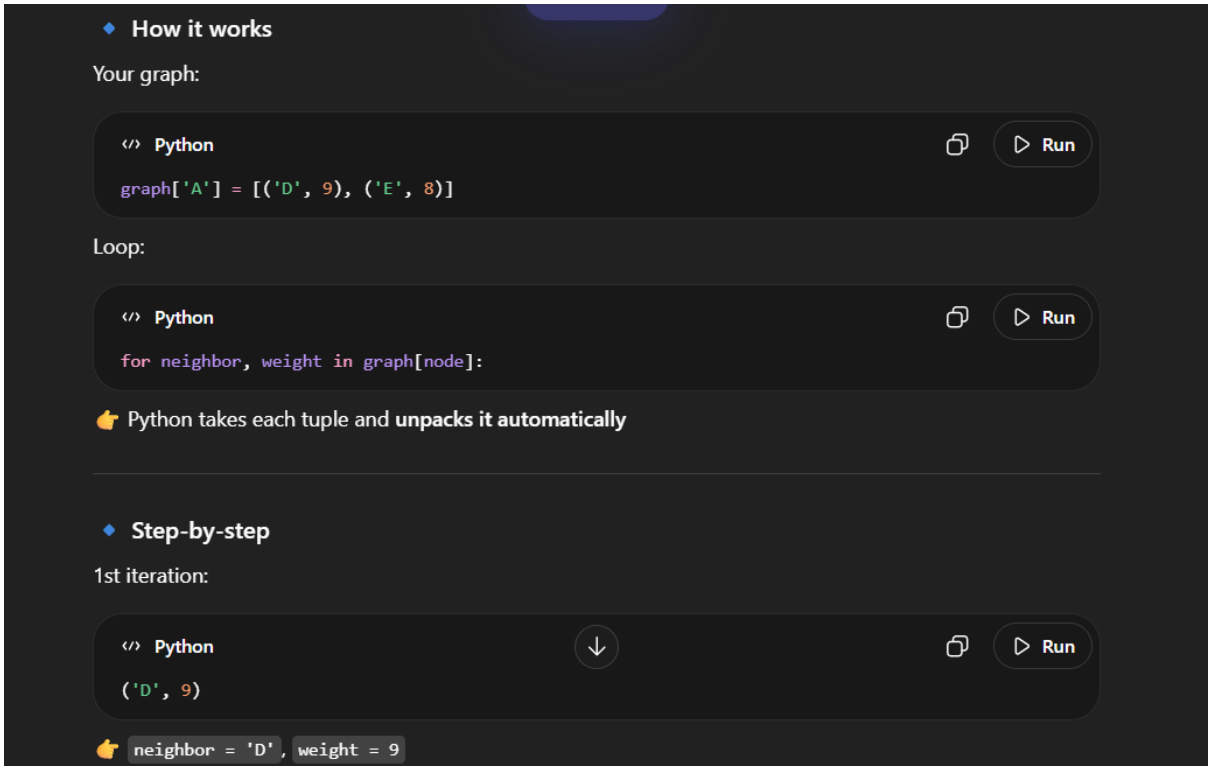
- Checks if we've **found the goal** — if yes, prints success and exits immediately

Expanding Neighbors

```python
            for neighbor, weight in graph[node]:
                if neighbor not in visited:
                    pq.put((weight, neighbor))
```

- Loops through all **neighbors of the current node**
- Skips already-visited neighbors
- Adds unvisited neighbors to the PQ with their **heuristic weight**

Example: Expanding node `A` adds `(9, D)` and `(8, E)` to the PQ



MAZE BFS

We explore the maze by always moving toward the cell **closest to the goal**, and stop when we reach it, then backtrack to get the path.

🔹 Idea of solving maze (background)

Maze = grid

- `0` → path
- `1` → wall
- Start → `(0,0)`
- Goal → `(4,4)`

👉 Problem:

> Find a path from **start** → **goal** without hitting walls.

🔹 Why BFS (basic idea)

BFS (Breadth-First Search):

> Explore level by level (all nearby cells first)

✔️ Guarantees **shortest path**
❌ But explores too many unnecessary nodes

🔹 What your code is doing (important ⚡)

This is actually **Best-First Search (not pure BFS)**

👉 Difference:

- BFS → no guidance
- Best-First → uses **heuristic (hint toward goal)**

Short and clear 👇

🔹 Why `class Node` is used

👉 To store **multiple things together for each position**

Instead of just `(x, y)`, we need:

- position
- parent (for path)
- cost values (`g`, `h`, `f`)

🔹 With class ✅

Each node stores:

```python
Node(
  position = (2,3),
  parent = previous_node,
  g, h, f
)
```

🔹 Main reasons

1. **Path tracing**

```python
current_node.parent
```

👉 go back to start

2. **Store heuristic values**

(`g`, `h`, `f`)

3. **Priority queue sorting**

```python
def __lt__(self, other):
    return self.f < other.f
```

👉 decides which node is better

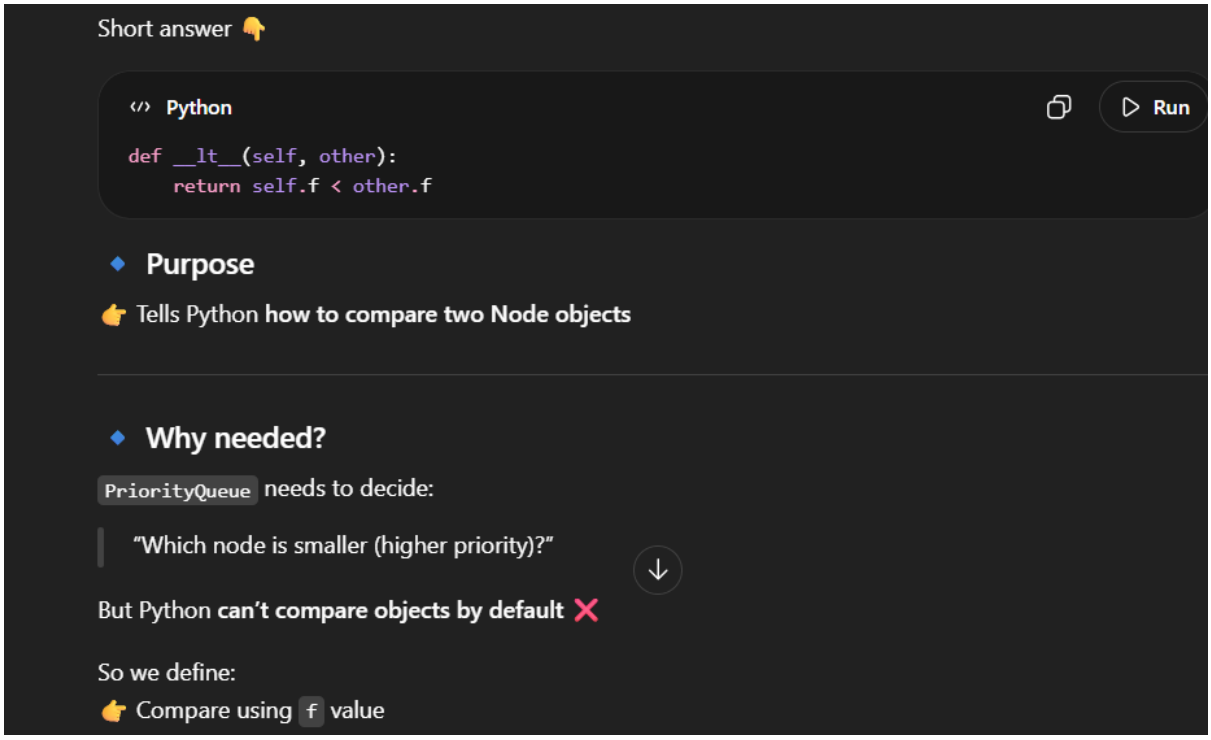
🔹 Line 1

```python
from queue import PriorityQueue
```

👉 Imports PriorityQueue so we can always pop the most promising node first

🔹 Line 2

```python
from queue import PriorityQueue
class Node:
    def __init__(self, position, parent=None):
        self.position = position
        self.parent = parent
        self.g = 0  # cost from start node to current node
        self.h = 0  # heuristic estimate of the cost from current node
to end node
        self.f = 0  # total cost
```



👉 **Yes — but specifically: it's the "priority cost" used for decision**

🔹 In your code

```python
new_node.f = new_node.h
```

👉 So here:

> **f = h (heuristic only)**

✔ Meaning:

- `f` = how close node is to goal
- Lower `f` → better node

🔹 In general (important)

Usually:

```python
f = g + h
```

- `g` → cost from start
- `h` → estimated cost to goal
- `f` → total cost

🔹 But in YOUR code

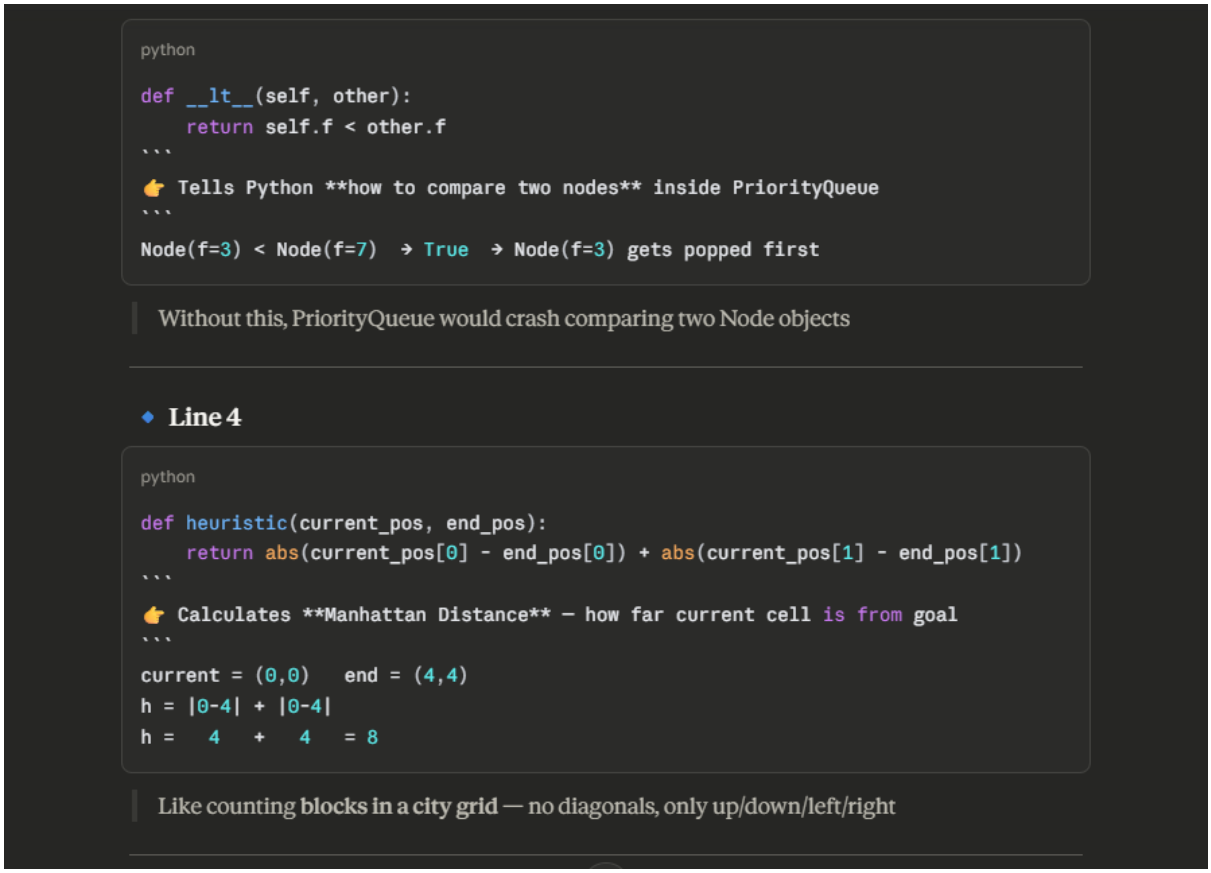
> It's **Greedy Best-First Search**

So:

- ignores `g`
- only uses `h`

🔹 One-line

> `f` is the value used to decide priority — in your case, it's just the heuristic (distance to goal).



Manhattan distance = how far current cell is from goal. Like finding heuristic .

abs ensures distance is always positive when calculating how far we are from the goal.

```
def best_first_search(maze, start, end):
    rows, cols = len(maze), len(maze[0])
    start_node = Node(start)
    end_node = Node(end)
    frontier = PriorityQueue()
    frontier.put(start_node)
    visited = set()
```

🔹 Line 5

python

```
rows, cols = len(maze), len(maze[0])
```

```
👉 Gets the **size of the maze** to check boundaries later
```

```
maze is 5x5
rows = 5
cols = 5
```

Python Run

```
rows, cols = len(maze), len(maze[0])
```

🔹 Why `len(maze[0])`?

👉 `maze` = list of rows

👉 `maze[0]` = **first row**

🔹 So:

- `len(maze)` → number of rows
- `len(maze[0])` → number of columns

Get Plus ×

Python Run

```
maze = [
 [0,0,1],
 [0,1,0]
]
```

- `len(maze)` = 2 rows
- `len(maze[0])` = 3 columns

🔹 Why needed?

To check boundaries:

Python Run

```
0 <= col < cols
```

🔹 Line 6

```python
start_node = Node(start)
end_node = Node(end)
```

👉 Creates Node objects for **start and end positions**

```
start_node.position = (0,0)
end_node.position   = (4,4)
```

🔹 Line 7

```python
frontier = PriorityQueue()
frontier.put(start_node)
visited = set()
```

👉 Sets up the frontier queue and visited tracker, then **adds start node**

```
frontier = [(f=0, Node(0,0))]
visited  = {}
```

👉 Keeps looping and **pops the node with lowest f score** each time

🔹 Line 8

```python
while not frontier.empty():
    current_node = frontier.get()
    current_pos = current_node.position
```

👉 Keeps looping and **pops the node with lowest f score** each time

```
frontier has → [Node(f=2), Node(f=5), Node(f=6)]
frontier.get() → picks Node(f=2)   ← most promising
current_pos  → (1,1)
```

👉 If goal reached, **traces back through parents** to rebuild the path

🔹 Line 9

```python
if current_pos == end:
    path = []
    while current_node:
        path.append(current_node.position)
        current_node = current_node.parent
    return path[::-1]
```

👉 If goal reached, **traces back through parents** to rebuild the path

```
Goal node  (4,4) → parent (3,4)
           (3,4) → parent (2,4)
           (2,4) → parent (1,4)  ... and so on

path (reversed) = [(0,0),(1,0),(1,1),(1,2),(1,3),(1,4),(2,4),(3,4),(4,4)]
```

> Like following **breadcrumbs back home**

Short 👇

Python Run

```
while current_node:
```

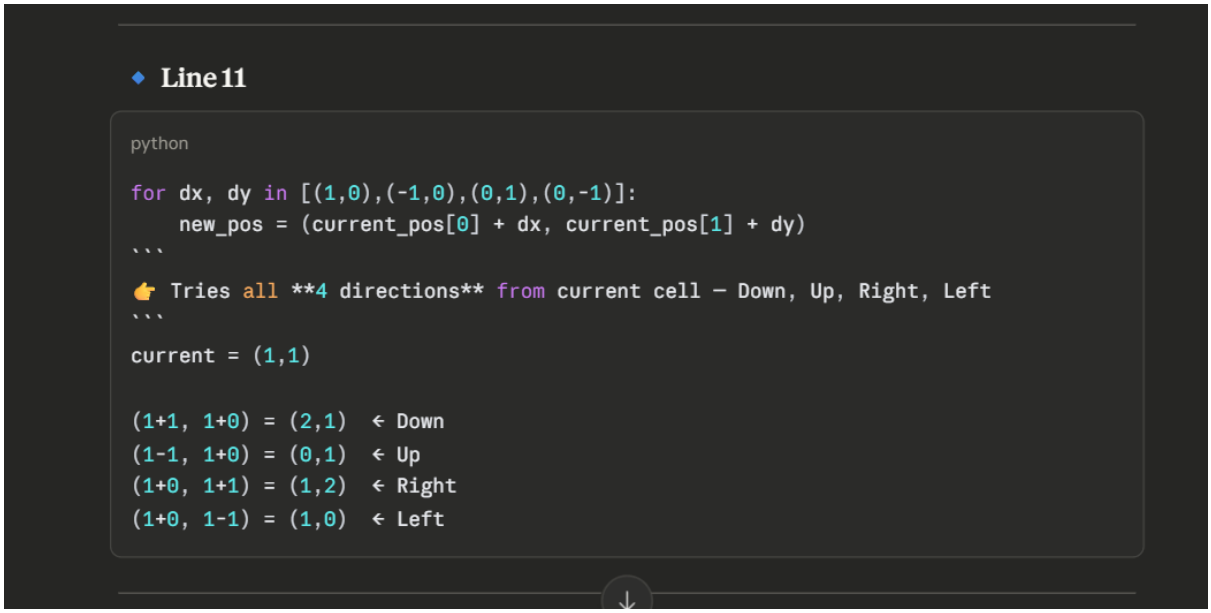
🔹 What is it checking?

👉 It checks:

> "Is `current_node` NOT None?"

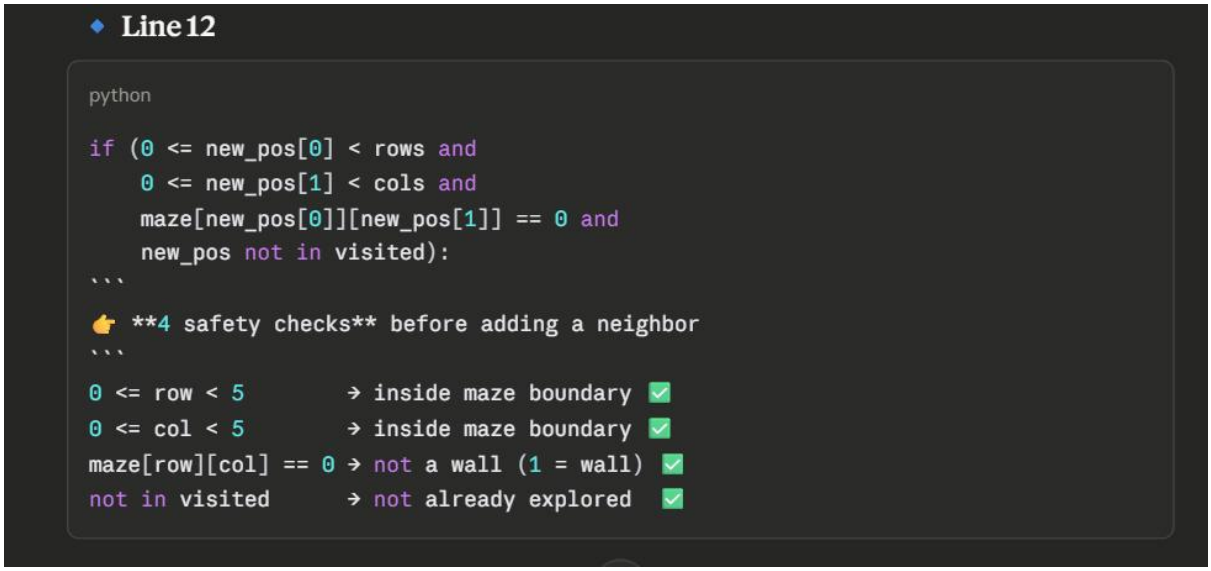
🔹 How it works

- `current_node` is an object → **True**
- When it becomes `None` → **False** → **loop stops**



After marking as visited , we travel up down , right , left

After that we do 4 checks of maze.



Now we create neighbour node

🔹 Line 13

python

```
new_node = Node(new_pos, current_node)
new_node.g = current_node.g + 1
new_node.h = heuristic(new_pos, end)
new_node.f = new_node.h
frontier.put(new_node)
visited.add(new_pos)
```

👉 Creates neighbor node, **calculates its scores**, adds to frontier

```
current = (1,1),  neighbor = (1,2),  end = (4,4)

g = 1 + 1 = 2                        ← one more step taken
h = |1-4| + |2-4| = 3 + 2 = 5        ← Manhattan distance
f = h = 5                             ← BFS only uses h to decide

frontier.put(Node(f=5, pos=(1,2)))
visited.add((1,2))
```

🔹 Line 14

python

```
maze = [
[0,0,1,0,0],
[0,0,0,0,0],
[0,0,1,0,1],
[0,0,1,0,0],
[0,0,0,1,0]
]
```

👉 The actual maze — **0 = open path, 1 = wall**

```
Visual:
. . # . .
. . . . .
. . # . #
. . # . .
. . . # .
Start=(0,0) top-left   Goal=(4,4) bottom-right
```

Maze Multi-Goal Best-First Search

🔹 Overall Code At A Glance

```
START (0,0)
    ↓
PriorityQueue — always pops cell closest to ANY goal
    ↓
Heuristic = min Manhattan Distance to all goals
    ↓
Expand 4 neighbors → calculate h → push to frontier
    ↓
First goal hit? → trace parent{} dictionary → return path
    ↓
Goals = (4,4) or (2,4) — whichever comes first WINS
```

Key difference from previous codes:

Previous Maze Code	This Code
One fixed goal Node	Multiple goals in a list
`parent` stored inside Node object	`parent` stored in a **dictionary**
h = distance to one end	h = **minimum** distance to any goal

🔹 Line 2

```python
def heuristic(pos, goals):
    return min(abs(pos[0]-g[0]) + abs(pos[1]-g[1]) for g in goals)
```

👉 Calculates Manhattan Distance to **every goal** and returns the **smallest one**

```
pos   = (1,2)
goals = [(4,4), (2,4)]

distance to (4,4) = |1-4| + |2-4| = 3+2 = 5
distance to (2,4) = |1-2| + |2-4| = 1+2 = 3

min(5, 3) = 3  ← this cell is 3 steps from nearest goal
```

> Like a GPS that checks **all destinations** and picks the closest one

🔹 Line 3

python

```
def best_first_multi_goal(maze, start, goals):
    rows = len(maze)
    cols = len(maze[0])
```

👉 Defines the function and gets **maze dimensions** for boundary checking

```
maze is 5x5
rows = 5
cols = 5
```

🔹 Line 4

python

```
frontier = PriorityQueue()
frontier.put((0, start))
visited = set()
```

👉 Sets up frontier and visited tracker, then **inserts start with cost 0**

```
frontier = [(0, (0,0))]
visited  = {}
```

🔹 Line 5 ⭐ (New Concept)

python

```
parent = {start: None}
```

👉 A **dictionary** that remembers which cell we came from — used to retrace path

```
parent = {(0,0): None}          # start has no parent

# as we explore...
parent = {
    (0,0): None,
    (1,0): (0,0),   ← we reached (1,0) FROM (0,0)
    (0,1): (0,0),   ← we reached (0,1) FROM (0,0)
    (1,1): (1,0),   ← we reached (1,1) FROM (1,0)
}
```

> Like leaving **sticky notes** on each cell saying "I came from here"

🔹 Line 6

```python
while not frontier.empty():
    _, current = frontier.get()
```

👉 Keeps looping and **pops the cell with lowest heuristic** each time

```
frontier has → [(3,(1,2)), (5,(0,1)), (6,(2,0))]
frontier.get() → pops (3, (1,2))

_ = 3          ← cost (ignored with _)
current = (1,2)
```

> `_` means "I don't need this value" — we only care about the position

```python
if current in goals:
    path = []
    while current:
        path.append(current)
        current = parent[current]
    path.reverse()
    return path
```

👉 If current cell is **any goal**, traces back through parent dictionary

```
goals   = [(4,4), (2,4)]
current = (2,4)  → YES it's a goal! 🎯

# trace back using parent{}
(2,4) → parent[(2,4)] = (2,3)
(2,3) → parent[(2,3)] = (2,2)
(2,2) → parent[(2,2)] = (1,2)
(1,2) → parent[(1,2)] = (1,1)
(1,1) → parent[(1,1)] = (0,1)
(0,1) → parent[(0,1)] = (0,0)
(0,0) → parent[(0,0)] = None → STOP
```

🔹 Line 11 ⭐ (Key Difference)

```python
parent[next_pos] = current
frontier.put((heuristic(next_pos, goals), next_pos))
visited.add(next_pos)
```

👉 Records parent, calculates h to **nearest goal**, pushes to frontier

```
current  = (2,2)
next_pos = (2,3)
goals    = [(4,4), (2,4)]

parent[(2,3)] = (2,2)     ← remember we came from (2,2)

h to (4,4) = |2-4|+|3-4| = 2+1 = 3
h to (2,4) = |2-2|+|3-4| = 0+1 = 1
min = 1

frontier.put((1, (2,3)))  ← very promising! only 1 step from a goal
```

```
61  maze = [
62      [0,0,0,1,0],
63      [1,0,0,1,0],
64      [0,0,0,0,0],
65      [0,1,1,0,1],
66      [0,0,0,0,0]
67      ]
68
69      start = (0,0)
70
71      goals = [(4,4),(2,4)]
72
73      path = best_first_multi_goal(maze, start, goals)
74
75      print("Path to nearest goal:", path)
```

GREEDY BEST FIRST SEARCH:

```python
from queue import PriorityQueue

graph = {
    'A': {'B':2,'C':1},
    'B': {'D':4,'E':3},
    'C': {'F':1,'G':5},
    'D': {'H':2},
    'E': {},
    'F': {'I':6},
    'G': {},
    'H': {},
    'I': {}
}

heuristic = {
    'A':7,'B':6,'C':5,'D':4,'E':7,
    'F':3,'G':6,'H':2,'I':0
}
```

Whenever we do write bfs function , first we write priority queue , then do frontier.put . then do visited = set() then write drictionary to trace parents at end .

```python
def greedy_bfs(graph, start, goal):
    frontier = PriorityQueue()
    frontier.put((heuristic[start], start))
    visited = set()
    came_from = {start: None}

    while not frontier.empty():
        h, current_node = frontier.get()

        if current_node not in visited:          # ✅ simple check
            print(current_node, end=" ")
            visited.add(current_node)
```

then we start while loop till frotniter is not empty. We then get the most promising node through frontier.get. then we check if its been visited , if not , then we add to visited . and then check if it's the goal , if goal , then we trace path of parent. We reverse the path and then return it

```
            if current_node == goal:
                path = []
                while current_node is not None:
                    path.append(current_node)
                    current_node = came_from[current_node]
                path.reverse()
                print("\nGoal found. Path:", path)
                return
```

If its not the goal , then we expand the nodes to its neighbours

```
            for neighbor in graph[current_node]:
                if neighbor not in visited:
                    came_from[neighbor] = current_node
                    frontier.put((heuristic[neighbor], neighbor))

    print("\nGoal not found")

greedy_bfs(graph, 'A', 'I')
```

🔹 The Graph Structure

```python
graph = {
    'A': {'B':2, 'C':1},   # A connects to B and C
    'B': {'D':4, 'E':3},   # B connects to D and E
    'C': {'F':1, 'G':5},   # C connects to F and G
}
```

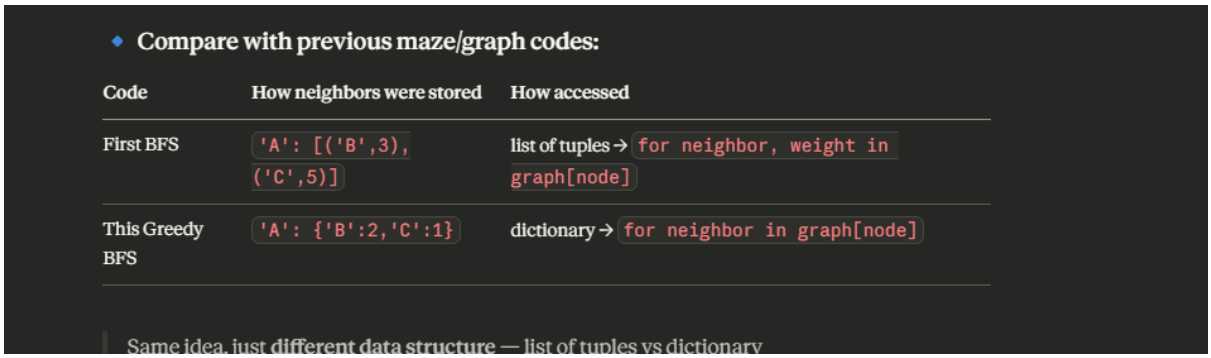
👉 Each node's value is a **dictionary** — keys are neighbors, values are edge costs

🔹 What happens when we write:

```python
for neighbor in graph[current_node]:
```
```
```
```
current_node = 'A'

graph['A']        → {'B':2, 'C':1}   ← full dictionary
for neighbor in   → loops over KEYS only

iteration 1 → neighbor = 'B'
iteration 2 → neighbor = 'C'
```

↓

When you loop over a dictionary in Python, you automatically get the **keys only** — not the values

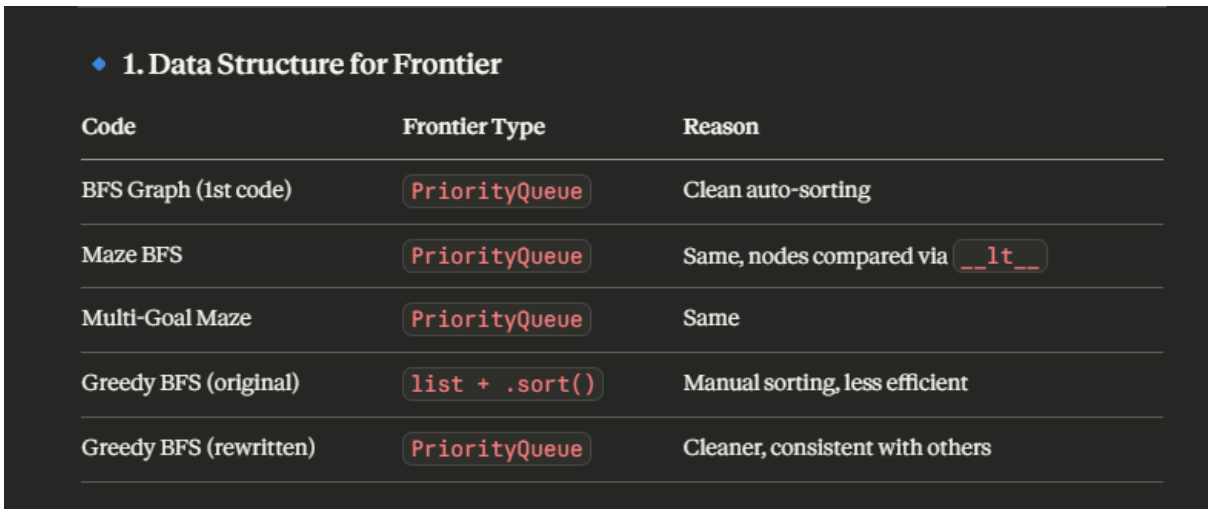


🔹 **Compare with previous maze/graph codes:**

Code	How neighbors were stored	How accessed
First BFS	`'A': [('B',3), ('C',5)]`	list of tuples → `for neighbor, weight in graph[node]`
This Greedy BFS	`'A': {'B':2,'C':1}`	dictionary → `for neighbor in graph[node]`

> Same idea, just different data structure — list of tuples vs dictionary

CODE DIFFERENCES:



🔹 **1. Data Structure for Frontier**

Code	Frontier Type	Reason
BFS Graph (1st code)	`PriorityQueue`	Clean auto-sorting
Maze BFS	`PriorityQueue`	Same, nodes compared via `__lt__`
Multi-Goal Maze	`PriorityQueue`	Same
Greedy BFS (original)	`list + .sort()`	Manual sorting, less efficient
Greedy BFS (rewritten)	`PriorityQueue`	Cleaner, consistent with others

🔹 **2. How frontier.put() differs**

Code	frontier.put()	Reason
BFS Graph (1st code)	`pq.put((weight, node))`	weight = neighbor's edge cost
Maze BFS	`frontier.put(node_object)`	entire Node object pushed, PQ uses `__lt__` to compare f values
Multi-Goal Maze	`frontier.put((0, start))` then `frontier.put((h, next_pos))`	tuple of (h, position)
Greedy BFS	`frontier.put((heuristic[node], node))`	only h value, edge cost ignored

🔹 3. How frontier.get() unpacks

Code	frontier.get()	Reason
BFS Graph (1st code)	`cost, node = pq.get()`	unpacks (cost, node) tuple
Maze BFS	`current_node = frontier.get()`	gets full Node object, position accessed via `current_node.position`
Multi-Goal Maze	`_, current = frontier.get()`	`_` ignores h value, only position needed
Greedy BFS	`h, current_node = frontier.get()`	h unpacked but never used after

🔹 4. How neighbors are stored in graph

Code	Graph Structure	How Neighbors Accessed
BFS Graph (1st code)	`'A': [('B',3),('C',5)]`	`for neighbor, weight in graph[node]`
Greedy BFS	`'A': {'B':2,'C':1}`	`for neighbor in graph[node]`
Maze codes	No graph dict — uses maze grid `maze[row][col]`	`for dx,dy in [(1,0),(-1,0),(0,1),(0,-1)]`

🔹 5. How path is retraced

Code	Path Tracing Method	Reason
BFS Graph (1st code)	❌ No path retracing — only prints visited nodes	simple traversal, no path needed
Maze BFS	`Node.parent` → follow chain of Node objects	parent stored inside Node class
Multi-Goal Maze	`parent{}` dictionary → `parent[pos] = current`	no Node class, dict used instead
Greedy BFS	`came_from{}` dictionary → `came_from[node] = current`	same as above, different name

🔹 6. How visited is checked

Code	Visited Check Style
BFS Graph (1st code)	`if node not in visited:` — everything inside the if block
Maze BFS	`if next_pos not in visited:` before pushing to frontier
Multi-Goal Maze	`if next_pos not in visited:` before pushing + `visited.add(next_pos)` when pushing
Greedy BFS (original)	`if current in visited: continue` — skip with continue
Greedy BFS (rewritten)	`if current_node not in visited:` — cleaned up, consistent ✅

🔹 7. Heuristic Calculation

Code	Heuristic	How Calculated
BFS Graph (1st code)	Edge weight only	taken directly from graph tuple
Maze BFS	Manhattan Distance	calculated live → `abs(r1-r2) + abs(c1-c2)`
Multi-Goal Maze	Min Manhattan Distance	`min(distance to each goal)` calculated live
Greedy BFS	Pre-defined h values	looked up from `heuristic{}` dictionary

Difference between Best First Search & Greedy Best First Search:
BFS uses an evaluation function f(n) to prioritize nodes, which **can include factors like cost g(n)) or heuristic estimates h(n))**. It is flexible but not guaranteed to be complete or optimal unless the evaluation function is carefully designed. Its behavior depends on how f(n) is defined. A specific variant of BFS **(Greedy BFS) uses only the heuristic function h(n)** to greedily select the node

closest to the goal. It ignores the cost to reach the node g(n)), making it fast but not guaranteed to be complete or optimal. It can produce suboptimal solutions or get stuck in loops.

A* Search

2.3 A* search (A- Star Search)

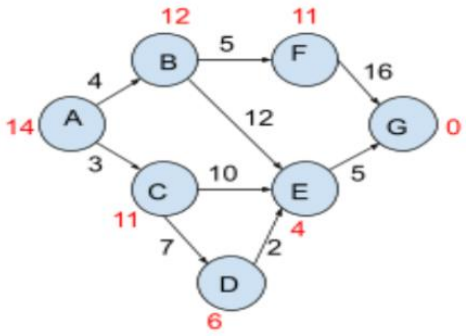
The A* algorithm is a widely used informed search algorithm that combines the strengths of uniform-cost search (using g(n), the cost to reach the current node) and greedy best-first search (using h(n), the heuristic estimate of the cost to the goal).

Evaluation function f(n) = g(n) + h(n)

g(n) = cost **so far** to reach n
h(n)= estimated cost from n to goal

Graph Structure: The graph is represented as a dictionary, where each key is a parent node, and the value is again a dictionary which contains neighbours of the key and the cost required to reach that key.

```
from queue import PriorityQueue
# Graph with different edge costs
graph = {
    'A': {'B': 4, 'C': 3},
    'B': {'E': 12, 'F': 5},
    'C': {'D': 7, 'E': 10},
    'D': {'E': 2},
    'E': {'G': 5},
    'F': {'G': 16},
    'G': {},
}

heuristic = {'A': 14,'B': 12,'C': 11,'D': 6,'E': 4,'F': 11,'G': 0 }
```



A* Function: a_star(graph, start, goal)
The function performs A* search on the given tree (graph) starting from the start node and search for the goal node.

Core Idea

	Greedy BFS	A*
Formula	`f(n) = h(n)` only	`f(n) = g(n) + h(n)`
Uses edge costs?	❌ Ignores completely	✅ Uses to calculate g(n)
Optimal path?	❌ Not guaranteed	✅ Always optimal
Speed	Faster	Slightly slower

Simple Example Why Greedy Fails

```
A → B (cost 1, h=10)
A → C (cost 100, h=1)

Greedy picks C  (h=1 looks closer)  → total cost = 100 ❌
A*     picks B  (f=1+10=11 vs 101)  → total cost = 1   ✅
```

A* Search — Full Code Walkthrough

🔹 Overall Code At A Glance

```
START (A)
    ⬇
Frontier (list) — sorted by f(n) = g(n) + h(n)
    ⬇
g(n) = actual cost from start to current node
h(n) = pre-defined heuristic to goal
f(n) = g(n) + h(n) = total estimated cost
    ⬇
Expand neighbor with lowest f → update g_costs{}
    ⬇
Goal found? → trace came_from{} → return path
```

🔹 Why people write `0 +`

- Makes it **clear this is** `g + h`
- Keeps formula **consistent** when you later add `g` for other nodes:

Python Run

```python
new_f = g_cost + heuristic[next_node]
```

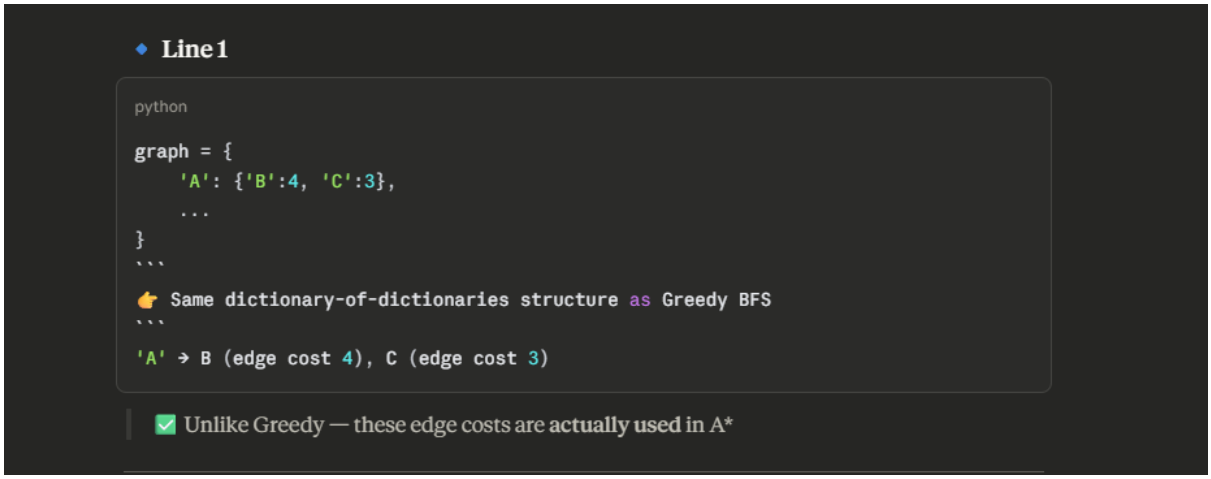
🔹 One-line

> Omitting `0 +` doesn't break the code; it's just **explicit notation for f = g + h.**

```python
from queue import PriorityQueue

graph = {
    'A': {'B':4,'C':3},
    'B': {'E':12,'F':5},
    'C': {'D':7,'E':10},
    'D': {'E':2},
    'E': {'G':5},
    'F': {'G':16},
    'G': {}
}

heuristic = {
    'A':14,'B':12,'C':11,'D':6,'E':4,'F':11,'G':0
}

def a_star(graph, start, goal):
    frontier = PriorityQueue()
    frontier.put((heuristic[start], start))   # (f value, node)
    visited = set()
    g_costs = {start: 0}
    came_from = {start: None}
```

```
    came_from = {start: None}

    while not frontier.empty():
        f, current_node = frontier.get()      # pop lowest f node

        if current_node not in visited:
            print(current_node, end=" ")
            visited.add(current_node)

            if current_node == goal:
                path = []
                while current_node is not None:
                    path.append(current_node)
                    current_node = came_from[current_node]
                path.reverse()
                print("\nGoal found with A*. Path:", path)
                return
```

```python
            for neighbor,cost in graph[current_node].items():

                new_g_cost=g_costs[current_node]+cost
                f_cost=new_g_cost+heuristic[neighbor]

                if neighbor not in g_costs or new_g_cost<g_costs[neighbor]:

                    g_costs[neighbor]=new_g_cost
                    came_from[neighbor]=current_node
                    frontier.append((neighbor,f_cost))

        print("\nGoal not found")


print("\nFollowing is the A* Search:")
a_star(graph,'A','G')
```



G cost is new data structure dictionary

1 line

TXT

why did we do this here and did this for neighbor in graph[current_node]: in greedy when we have used dictionry

Great observation! Here is why:

🔹 In Greedy BFS

```python
for neighbor in graph[current_node]:
```
👉 We only needed the **neighbor name** — edge cost was **completely ignored**
```
graph['A'] = {'B':4, 'C':3}

for neighbor in graph['A']:
    → neighbor = 'B'   ← just the key
    → neighbor = 'C'   ← just the key

cost? → don't care ❌
```

🔹 In A*

```python
for neighbor, cost in graph[current_node].items():
```
👉 We needed **both** neighbor name AND edge cost to calculate g(n)
```
graph['A'] = {'B':4, 'C':3}

for neighbor, cost in graph['A'].items():
    → neighbor='B', cost=4   ← key AND value
    → neighbor='C', cost=3   ← key AND value

new_g_cost = g_costs[current] + cost   ← cost is used here ✅
```

🔹 The difference is `.items()`

	Code	Gets
Greedy	`for neighbor in graph[node]`	keys only
A*	`for neighbor, cost in graph[node].items()`	keys + values

```python
d = {'B':4, 'C':3}

for x in d:          → 'B', 'C'          # keys only
for x in d.items():  → ('B',4), ('C',3)  # key-value pairs
```

🔹 Why Recalculate Cost Every Time

```
Start = A

Path 1:  A → C → D → E
g cost:  0 + 3 + 7 + 2 = 12  ✅ cheaper!

Path 2:  A → C → E
g cost:  0 + 3 + 10 = 13  ❌ expensive
```

👉 Node **E can be reached multiple ways** — we recalculate each time to find the **cheapest route**

route

```python
if neighbor not in g_costs or new_g_cost < g_costs[neighbor]:
    g_costs[neighbor] = new_g_cost   # update only if cheaper
```

```
First time  E reached → g = 13   stored ✅
Second time E reached → g = 12   12 < 13 → UPDATE ✅
Third time  E reached → g = 15   15 > 12 → SKIP  ❌
```

> This is exactly why A* is **optimal** — it never settles for an expensive path when a cheaper one exists ✅

Great question! Here is why:

🔹 The Full Condition

```python
if neighbor not in g_costs or new_g_cost < g_costs[neighbor]:
```

👉 It has **two parts** joined by `or`

🔹 Part 1 — `neighbor not in g_costs`

```python
g_costs = {'A':0, 'C':3}   # only A and C visited so far

neighbor = 'D'   → D not in g_costs → FIRST TIME seeing D
```

> If we have **never seen this neighbor before** — just add it directly, no comparison needed

---

🔹 Part 2 — `new_g_cost < g_costs[neighbor]`

```python
g_costs = {'A':0, 'C':3, 'D':10}   # D already exists

neighbor = 'D', new_g_cost = 8
8 < 10 → cheaper path found → UPDATE ✅

neighbor = 'D', new_g_cost = 12
12 < 10 → more expensive → SKIP ❌
```

🔹 Why we need BOTH conditions

```python
# if we only wrote:
if new_g_cost < g_costs[neighbor]:
```

```
neighbor = 'D'
D not in g_costs        → g_costs['D'] doesn't exist
g_costs['D']            → 💥 KeyError CRASH!
```

> First visit → no key exists yet → **crash without the first check** Second visit → key exists → compare costs safely

🔹 Simple Summary

Situation	Which part handles it
Neighbor seen for first time	`neighbor not in g_costs`
Neighbor seen before, cheaper path found	`new_g_cost < g_costs[neighbor]`
Neighbor seen before, expensive path	neither → SKIP

Came from plays a major role here effectively updating the path if we get the cheaper alternative

```python
if neighbor not in g_costs or new_g_cost < g_costs[neighbor]:
    g_costs[neighbor] = new_g_cost      # update cost
    came_from[neighbor] = current_node  # update parent ✅
    frontier.put((f_cost, neighbor))    # push again
```

👉 **Three things update at once** — and `came_from` is the key one for path

---

🔹 Trace Example

```
First time E reached via C:
    g_costs['E']   = 13
    came_from['E'] = 'C'    ← E came from C

Second time E reached via D (cheaper!):
    new_g = 12 < 13
    g_costs['E']   = 12     ← cost updated
    came_from['E'] = 'D'    ← E now came from D ✅ OVERWRITTEN
```

🔹 Why came_from update matters

```
```

Before update → path would be:

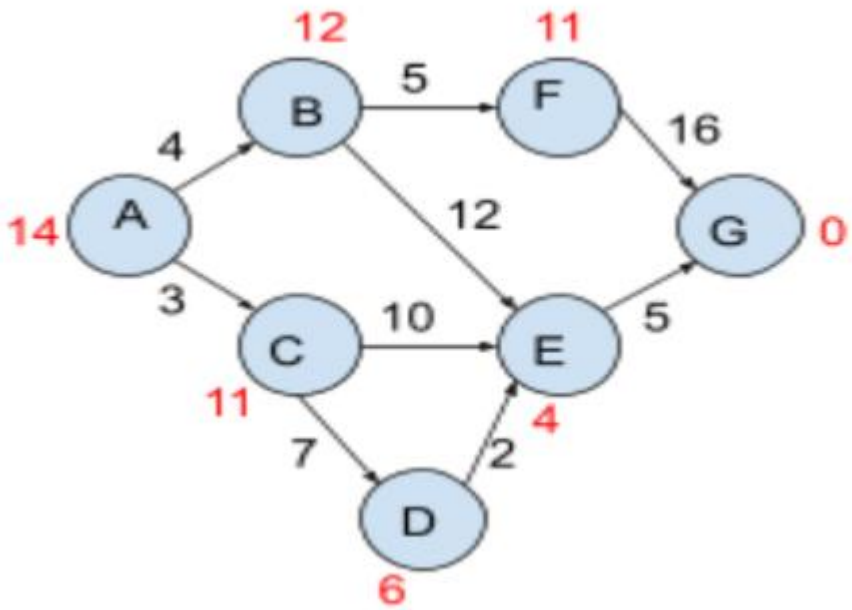
G ← E ← C ← A cost = 13+5 = 18 ❌

After update → path becomes:

G ← E ← D ← C ← A cost = 12+5 = 17 ✅

came_from[neighbor] = current_node **rewrites the breadcrumb** — so when we trace back at the end, we automatically follow the **cheapest route** ✅

EXAMPLE:




```
16
17    def a_star(graph, start, goal):
18        frontier = PriorityQueue()
19        frontier.put((heuristic[start], start))   # (f value, node)
20        visited = set()
21        g_costs = {start: 0}
22        came_from = {start: None}
23
```

Setup

```
graph edges:          heuristic values:
A→B(4)  A→C(3)        A=14  B=12  C=11
B→E(12) B→F(5)        D=6   E=4   F=11
C→D(7)  C→E(10)       G=0
D→E(2)
E→G(5)
F→G(16)
```

STEP 1 — Initialize

```
frontier.put((heuristic['A'], 'A'))
frontier.put((14, 'A'))

frontier  = [(14, 'A')]
g_costs   = {'A': 0}
came_from = {'A': None}
visited   = {}
```

STEP 2 — Pop A (f=14)

```
f, current = frontier.get() → f=14, current='A'
'A' not in visited ✅

print → A
visited = {'A'}

neighbors of A:
```

G cost is basically the cost so far to reach the node

Exactly ✅

- 🔹 `g` **cost in A***
- `g` = **actual cost from start to current node**
- Every time you move to a new node, you add the step cost

```
neighbors of A:
┌──────────────────────────────────────────┐
| neighbor B, edge cost = 4                |
|   new_g  = g_costs['A'] + 4 = 0+4 = 4    |
|   f      = 4 + h['B'] = 4+12 = 16        |
|   B not in g_costs → ADD ✅               |
|   g_costs['B']   = 4                     |
|   came_from['B'] = 'A'                   |
|   frontier.put((16, 'B'))                |
├──────────────────────────────────────────┤
| neighbor C, edge cost = 3                |
|   new_g  = 0+3 = 3                       |
|   f      = 3 + h['C'] = 3+11 = 14        |
|   C not in g_costs → ADD ✅               |
|   g_costs['C']   = 3                     |
|   came_from['C'] = 'A'                   |
|   frontier.put((14, 'C'))                |
└──────────────────────────────────────────┘

frontier  = [(14,'C'), (16,'B')]
g_costs   = {'A':0, 'B':4, 'C':3}
came_from = {'A':None, 'B':'A', 'C':'A'
```

STEP 3 — Pop C (f=14) ← lowest

```
f, current = frontier.get() → f=14, current='C'
'C' not in visited ✅

print → C
visited = {'A','C'}

neighbors of C:
┌──────────────────────────────────────────┐
| neighbor D, edge cost = 7                |
|   new_g  = g_costs['C'] + 7 = 3+7 = 10   |
|   f      = 10 + h['D'] = 10+6 = 16       |
|   D not in g_costs → ADD ✅               |
|   g_costs['D']   = 10                    |
|   came_from['D'] = 'C'                   |
|   frontier.put((16, 'D'))                |
├──────────────────────────────────────────┤
| neighbor E, edge cost = 10               |
|   new_g  = 3+10 = 13                     |
|   f      = 13 + h['E'] = 13+4 = 17       |
|   E not in g_costs → ADD ✅               |
|   g_costs['E']   = 13                    |
|   came_from['E'] = 'C'                   |
|   frontier.put((17, 'E'))                |
└──────────────────────────────────────────┘

frontier  = [(16,'B'), (16,'D'), (17,'E')]
g_costs   = {'A':0, 'B':4, 'C':3, 'D':10, 'E':13}
```

STEP 4 — Pop B (f=16)

```
f, current = frontier.get() → f=16, current='B'
'B' not in visited ✅

print → B
visited = {'A','C','B'}

neighbors of B:
┌──────────────────────────────────────┐
| neighbor E, edge cost = 12           |
|   new_g  = g_costs['B'] + 12 = 4+12 = 16|
|   f      = 16 + h['E'] = 16+4 = 20   |
|   E in g_costs, 16 > 13 → SKIP ❌    |
├──────────────────────────────────────┤
| neighbor F, edge cost = 5            |
|   new_g  = 4+5 = 9                   |
|   f      = 9 + h['F'] = 9+11 = 20    |
|   F not in g_costs → ADD ✅          |
|   g_costs['F']   = 9                 |
|   came_from['F'] = 'B'               |
|   frontier.put((20, 'F'))            |
└──────────────────────────────────────┘

frontier  = [(16,'D'), (17,'E'), (20,'F')]
```

After getting correct g cost , we update g cost and then the parent with current nide

```
39
40              for neighbor, cost in graph[current_node].items():
41                  new_g_cost = g_costs[current_node] + cost
42                  f_cost = new_g_cost + heuristic[neighbor]
43                  if neighbor not in g_costs or new_g_cost < g_costs[neighbor]:
44                      g_costs[neighbor] = new_g_cost
45                      came_from[neighbor] = current_node
46                      frontier.put((f_cost, neighbor))  # ✅ push (f, node)
47
48      print("\nGoal not found")
49
50  print("\nFollowing is the A* Search:")
51  a_star(graph, 'A', 'G')
52
```

Use Control + Shift + m to toggle the tab key moving focus. Alternatively, use esc then tab to move to the next interac

DYNAMIC A* SEARCGH:

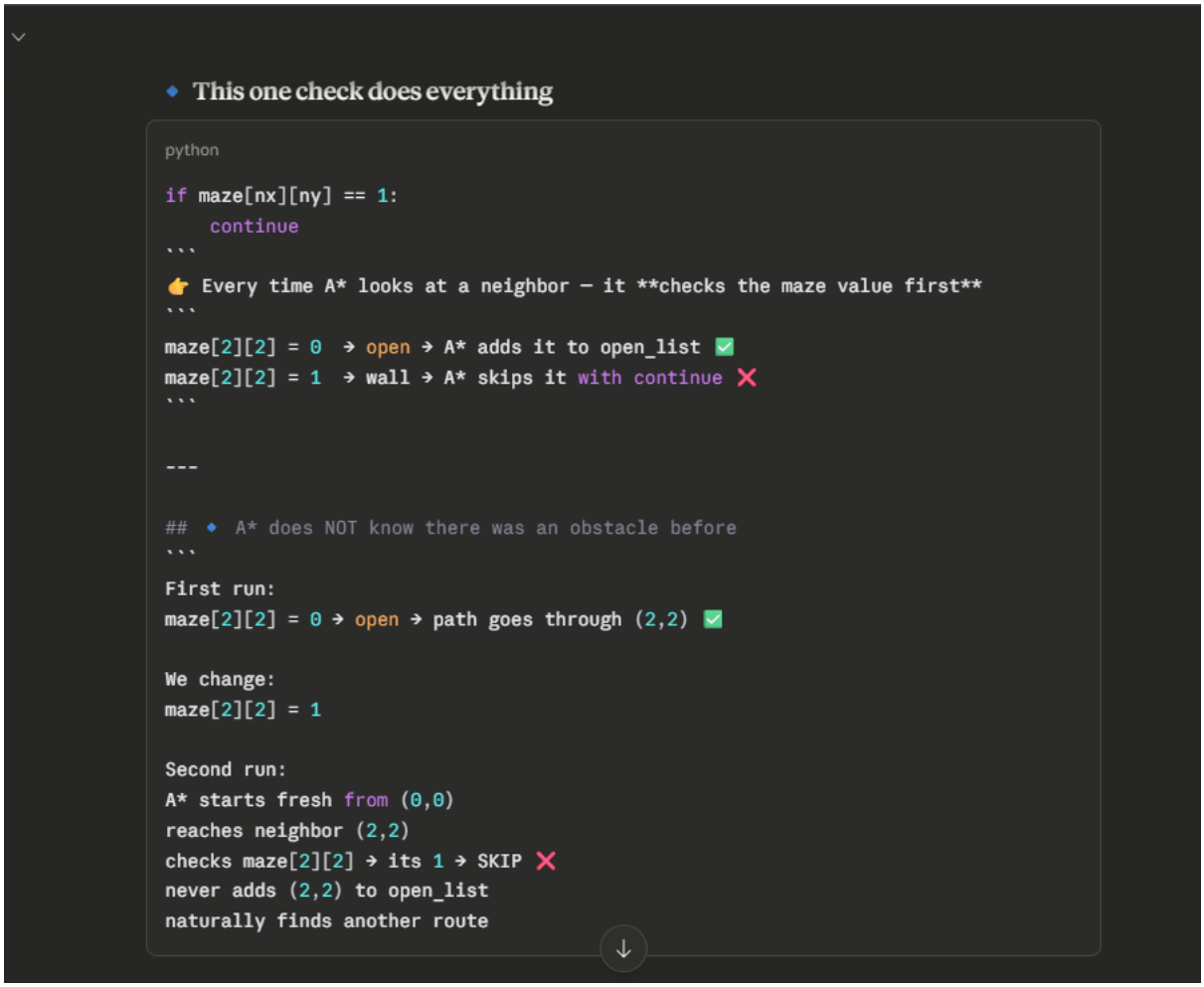
This program demonstrates Dynamic A* Search.

A* is first used to find the shortest path in the maze.

If an obstacle appears during navigation, the algorithm

recalculates a new optimal path from the current position

to the goal using the A* strategy.



🔹 Simple analogy

> Imagine you are walking to school Your normal route goes through a street Next day that street is **blocked by construction** You don't need anyone to tell you — you just **see the block and take another street** A* does the same — it just **reads the maze** and skips whatever is 1

🔹 Summary

	What happens
`maze[nx][ny] == 0`	open path → add to open_list
`maze[nx][ny] == 1`	wall → `continue` → skip completely
A* awareness	zero — it just reads numbers in the maze

> A* never remembers old paths — every run it **reads the maze fresh** and `== 1` means wall, that's it ✅

🔹 Simple analogy

> Imagine you are walking to school Your normal route goes through a street Next day that street is **blocked by construction** You don't need anyone to tell you — you just **see the block and take another street** A* does the same — it just **reads the maze** and skips whatever is 1

🔹 Summary

	What happens
`maze[nx][ny] == 0`	open path → add to open_list
`maze[nx][ny] == 1`	wall → `continue` → skip completely
A* awareness	zero — it just reads numbers in the maze

Dynamic Feature

Code	Static/Dynamic
Graph A*	❌ static graph, runs once
Dynamic A*	✅ maze changes → A* reruns automatically

python

```python
from queue import PriorityQueue

def heuristic(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def a_star(maze, start, goal):
    rows = len(maze)
    cols = len(maze[0])

    open_list = PriorityQueue()
    open_list.put((0, start))

    g_cost = {start: 0}
    parent = {start: None}
```

🔹 **Line 2**

python

```python
def heuristic(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])
```

👉 Manhattan Distance between any two cells

```
a = (0,0)   b = (4,4)
h = |0-4| + |0-4| = 8
```

🔹 Line 3

python

```
rows = len(maze)
cols = len(maze[0])
open_list = PriorityQueue()
open_list.put((0, start))
```

👉 Gets maze size and pushes start into open_list with f=0

```
rows = 5, cols = 5
open_list = [(0, (0,0))]
```

Short 👇

✦ Get Plus ×

‹/› Python

Run

```
open_list.put((0, start))
```

🔹 What is `0`?

👉 It is the **priority value (cost)** of the start node.

🔹 Why 0?

- At the beginning:
  - `g = 0` (no movement yet)
  - sometimes `f = 0` initially (or `g` only, depending on code)

👉 So we push:

> `(priority, node)`

🔹 Meaning

‹/› Python

Run

```
(0, start)
```

👉 "Start node has cost/priority = 0"

🔹 Line 4

python

```
g_cost = {start: 0}
parent = {start: None}
```

```
👉 Two dictionaries — g_cost tracks cheapest cost, parent tracks path
```

```
g_cost  = {(0,0): 0}
parent  = {(0,0): None}
```

🔹 Line 5

python

```
while not open_list.empty():
    _, current = open_list.get()
```

```
👉 Keeps looping and pops cell with lowest f each time
```

```
open_list = [(6,(2,0)), (7,(1,0)), (8,(0,1))]
open_list.get() → pops (6,(2,0))

_       = 6      ← f ignored
current = (2,0)
```

```python
while not open_list.empty():
    _, current = open_list.get()

    if current == goal:
        path = []
        while current:
            path.append(current)
            current = parent[current]
        path.reverse()
        return path
```

Theres no need to write visited.add here since g_ cost does our thing here after return path

🔹 In previous codes we had visited set

```python
visited = set()

if current_node not in visited:
    visited.add(current_node)
```

👉 We needed visited to **prevent revisiting nodes**

🔹 In Dynamic A* we have g_cost instead

```python
g_cost = {start: 0}

if next_node not in g_cost or new_g < g_cost[next_node]:
    g_cost[next_node] = new_g
```

👉 g_cost dictionary **already does the same job** — plus more

---

🔹 Line 7

```python
x, y = current
for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
    nx = x + dx
    ny = y + dy
    next_node = (nx, ny)
```

👉 Unpacks current cell and tries all 4 directions

```
current = (2,0)  x=2, y=0

Down  → (3,0)
Up    → (1,0)
Right → (2,1)
Left  → (2,-1)
```

Short 👇

```python
if maze[nx][ny] == 1:
    continue
```

🔹 What does it mean?

👉 If the next cell is a **wall (1)** → **skip it**

🔹 What `continue` does

👉 Skips the rest of the loop and moves to next direction

Example:

- Right = wall ❌ → skip
- Left = free ✅ → process it

🔹 One-line

> `continue` means: **ignore this blocked path and check the next possible move.**

by next direction you mean it goes to dx dy

Yes ✅

🔹 "Next direction" means next `(dx, dy)`

```python
for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
```

Yes ✅ (almost exactly)

🔹 Flow

Python

```python
if maze[nx][ny] == 1:
    continue
```

👉 If wall → skip ❌

👉 If NOT wall → execution continues ⬇️

Python

```python
new_g = g_cost[current] + 1
```

```python
        x, y = current
        for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
            nx = x + dx
            ny = y + dy
            next_node = (nx, ny)
            if 0 <= nx < rows and 0 <= ny < cols:
                if maze[nx][ny] == 1:
                    continue
                new_g = g_cost[current] + 1
                if next_node not in g_cost or new_g < g_cost[next_node]:
                    g_cost[next_node] = new_g
                    f_cost = new_g + heuristic(next_node, goal)
                    open_list.put((f_cost, next_node))
                    parent[next_node] = current

    return None
```

```python
42
43          for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
44
45              nx = x + dx
46              ny = y + dy
47              next_pos = (nx, ny)
48
49              if (0 <= nx < rows and
50                  0 <= ny < cols and
51                  maze[nx][ny] == 0 and
52                  next_pos not in visited):
53
54                  parent[next_pos] = current
55                  frontier.put((heuristic(next_pos, goals), next_pos))
56                  visited.add(next_pos)
57
58      return None
59
60
```

In best firsr search , we didn't do maze check of if separately, but here we did . khair we can do it in one also like bfs

python

```
if (0 <= new_pos[0] < rows and
    0 <= new_pos[1] < cols and
    maze[new_pos[0]][new_pos[1]] == 0 and
    new_pos not in visited):
```

👉 **All checks combined in ONE if block**

🔹 Dynamic A* Maze Code

python

```
if 0 <= nx < rows and 0 <= ny < cols:
    if maze[nx][ny] == 1:
        continue
```

👉 **Same checks but SPLIT into two separate if blocks**

🔹 They do exactly the same thing!

```
BFS Maze:
→ boundary check + wall check + visited → all one line

Dynamic A*:
→ boundary check first
→ then wall check separately with conti
```

🔹 Maze is just a Python list

python

```
maze = [
[0,0,0,0,0],
[0,1,1,1,0],
[0,0,0,1,0],
[1,1,0,0,0],
[0,0,0,1,0]
]
```

👉 It is a **normal Python list** — so we can change any value anytime

---

🔹 This one line changes the maze

INFORMED SEARCH CONCLUSION

🔹 Full Comparison — All Codes So Far

Frontier Type

Code	Frontier
BFS Graph	`PriorityQueue` — `(weight, node)`
Maze BFS	`PriorityQueue` — full Node object
Multi-Goal Maze	`PriorityQueue` — `(h, position)`
Greedy BFS	`PriorityQueue` — `(h[node], node)`
Graph A*	`PriorityQueue` — `(f, node)`
Dynamic A*	`PriorityQueue` — `(f, node)`

Heuristic

Code	Type
BFS Graph	edge weight only
Maze BFS	Manhattan — calculated live
Multi-Goal	min Manhattan to all goals
Greedy BFS	pre-defined dict lookup
Graph A*	pre-defined dict lookup
Dynamic A*	Manhattan — calculated live

Special Feature

Code	Feature
Multi-Goal	multiple goals — nearest wins
Greedy BFS	ignores edge cost completely
Graph A*	updates cheaper path via g_costs
Dynamic A*	maze changes mid-run → reruns A*

Path Tracing

Code	Method
BFS Graph	❌ no path tracing
Maze BFS	`Node.parent` chain
Multi-Goal Maze	`parent{}` dictionary
Greedy BFS	`came_from{}` dictionary
Graph A*	`came_from{}` dictionary
Dynamic A*	`parent{}` dictionary

```
    return None

maze = [
[0,0,0,0,0],
[0,1,1,1,0],
[0,0,0,1,0],
[1,1,0,0,0],
[0,0,0,1,0]
]
start = (0,0)
goal  = (4,4)

path = a_star(maze, start, goal)
print("Initial Path:", path)

maze[2][2] = 1
print("Obstacle added at (2,2). Recalculating...")

path = a_star(maze, start, goal)
print("New Path:", path)
```

🔹 Line 10 ⭐ The Dynamic Part

````
python

maze[2][2] = 1
path = a_star(maze, start, goal)
```
👉 Blocks cell (2,2) and reruns A* on updated maze
```
Before:              After:
0 0 0 0 0            0 0 0 0 0
0 1 1 1 0            0 1 1 1 0
0 0 0 1 0            0 0 1 1 0  ← (2,2) blocked!
1 1 0 0 0            1 1 0 0 0
0 0 0 1 0            0 0 0 1 0

Old path used (2,2) → now wall ❌
A* finds new path avoiding (2,2) ✅
````

🔹 Paste-Ready Python Comments

LOCAL SEARCHES

🔹 What We Did So Far — Informed Searches

```
BFS → Greedy → A* → Dynamic A*
    ↓
All start from a point
All search toward a goal
All track the path taken
All use frontier/open_list
```

🔹 Local Searches — Key Difference

```
No frontier
No path tracking
No start→goal concept

Instead:
Start at any point
Look at neighbors
Move to better neighbor
Keep improving until best solution found
```

🔹 Local Search Family

Algorithm	Core Idea
Hill Climbing	always move to best neighbor, stop at peak
Simulated Annealing	like hill climbing but accepts bad moves sometimes
Genetic Algorithm	evolves a population of solutions
Beam Search	keeps k best states at each step

🔹 So when they say:

🔸 **Informed Search finds optimal path**
→ It finds the *best path* based on cost

🔸 **Local Search finds optimal solution**
→ It finds the *best state/answer*, but:

- Not necessarily the path taken to reach it
- It only cares about the final result

🔹 Key Difference (VERY IMPORTANT)

Type	Focus
Informed Search (A*, etc.)	Finds optimal **path**
Local Search (Hill Climbing, etc.)	Finds optimal **solution/state**

👉 Local search usually **does NOT store the path**, only the final answer.

🔹 Simple Analogy

Imagine climbing a mountain:

- **Informed Search**:
  "What is the *best route* to the top?" 🏞️
- **Local Search**:
  "Just get me to the *highest point* — I don't care how" 🪨

🔹 What is Beam Search?

Beam Search is a type of **informed search** where:

> Instead of exploring ALL nodes, we only keep the **best** `k` **nodes at each level.**

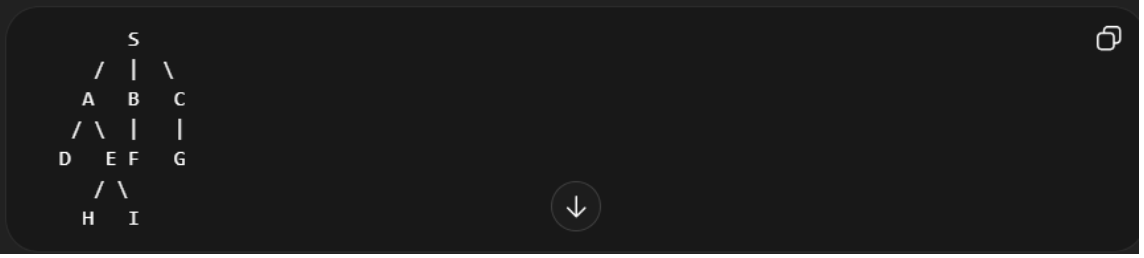
👉 `k` = **beam width**

🔹 Idea in one line

> "At every step, keep only the top `k` most promising paths, discard the rest."

🔹 Example Graph

Let's take a simple graph:



🔸 Heuristic values (h(n)) — lower is better:

Node	h(n)
A	4
B	2
C	5
D	6
E	1
F	3
G	7
H	0 (Goal)
I	2

🔹 Let's set:

👉 **Beam width (k) = 2**

🔹 Step-by-step working

✅ Step 1: Start at S

Expand S → get:

```
A(4), B(2), C(5)
```

👉 Keep best 2 (lowest h):

- ✔ B(2)
- ✔ A(4)
- ❌ C(5) (discarded)

✅ Step 2: Expand B and A

Expand B:

```
F(3)
```

Expand A:

```
D(6), E(1)
```

👉 All candidates now:

```
F(3), D(6), E(1)
```

👉 Pick best 2:

- ✔ E(1)
- ✔ F(3)
- ❌ D(6) (discarded)

✅ Step 3: Expand E and F

Expand E:

```
H(0), I(2)
```

Expand F:
(no children or irrelevant)

👉 Candidates:

```
H(0), I(2)
```

👉 Pick best 2:

✔ H(0) ← GOAL FOUND 🎯
✔ I(2)

🔹 Final Answer

Beam Search finds path:

```
S → A → E → H
```

🔹 VERY IMPORTANT Insight

⚠️ Beam Search **may NOT give optimal path**

Why?

Because it **discards nodes early** (like C or D), even if they could lead to a better solution later.

🔹 Key Characteristics

✔ Uses heuristic (like Greedy)
✔ Memory efficient
✔ Faster than BFS

❌ Not guaranteed optimal
❌ Can miss best path

🔹 Simple Analogy

Imagine you're job hunting:

- You only apply to **top 2 companies** at a time (beam width = 2)
- You ignore the rest

👉 Faster decision
👉 But maybe you missed a better company later

🔹 Overall Code At A Glance

```
START (S)
    ↓
beam = top k paths kept at each level
    ↓
Expand ALL paths in beam → get candidates
    ↓
Sort candidates by cost → keep only top k (beam_width)
    ↓
Discard rest → repeat with new beam
    ↓
Goal found inside beam → return path + cost
```

Key difference from all previous codes:

Previous Codes	Beam Search
Expand one node at a time	Expand **all k paths** at once
Keep all candidates	Keep only **top k candidates**
Never throws away paths	**Prunes** everything outside top k
Guaranteed optimal	❌ may miss optimal if pruned
Uses PriorityQueue	Plain **list**

🔹 Line 1

```python
graph = {
    'S': [('A',3), ('B',6), ('C',5)],
    ...
}
```

👉 Same adjacency list as our very first BFS code

```
S → A(3), B(6), C(5)
A → D(9), E(8)
L → []   ← goal, no neighbors
```

```python
def beam_search(start, goal, beam_width=2):
```

👉 Function takes start, goal and **beam_width — default is 2**

```
beam_width = 2 means:
→ at each level keep only 2 best paths
→ all others DISCARDED ❌
```

🔹 Line 3 ⭐ (New Concept)

```python
beam = [(0, [start])]
```

👉 Beam starts with **one path — just the start node with cost 0**

```
beam = [(0, ['S'])]
         ↑    ↑
       cost  path so far
```

> Unlike previous codes where we tracked single nodes — here we track **entire paths**

🔹 Line 4

```python
while beam:
    candidates = []
```

👉 Keeps looping while beam has paths, **candidates collects all expansions**

```
beam       = current top k paths
candidates = all possible next paths (before pruning)
```

🔹 Line 5 ⭐

```
python
for cost, path in beam:
    current = path[-1]
```
👉 Loops through every path in beam and gets **last node of each path**
```
beam = [(3, ['S','A']), (5, ['S','C'])]

path = ['S','A'] → current = 'A'   ← last element
path = ['S','C'] → current = 'C'   ← last element
```

> `path[-1]` means **last item in list** — that is where we currently are

---

🔹 Line 6

```
python
if current == goal:
    return path, cost
```
👉 If any path in beam **reached goal — return immediately**
```
current = 'L'
goal    = 'L'  → MATCH 🎯 → return path and cost
```

🔹 Line 5 ⭐

```
python
for cost, path in beam:
    current = path[-1]
```
👉 Loops through every path in beam and gets **last node of each path**
```
beam = [(3, ['S','A']), (5, ['S','C'])]

path = ['S','A'] → current = 'A'   ← last element
path = ['S','C'] → current = 'C'   ← last element
```

> `path[-1]` means **last item in list** — that is where we currently are

---

🔹 First: What is stored in `beam`?

Each element in `beam` is:

```python
(cost, path)
```

Example:

```python
(5, ['S', 'A', 'E'])
```

👉 Meaning:

- `cost = 5` → total cost so far
- `path = ['S', 'A', 'E']` → how we reached here

🔹 Why `current_node = path[-1]` ?

✅ **Because:**

> The **last element of the path is the current node**

Example:

```python
path = ['S', 'A', 'E']
```

- Start = S
- Then A
- Then E

👉 So **current node = E**

```python
current_node = path[-1]  # E
```

🔹 Now: How are we expanding?

This loop:

```python
for neighbor, edge_cost in graph.get(current_node, []):
```

👉 Means:

> "Find all nodes connected to the current node"

Example Graph:

```python
graph = {
    'E': [('H', 2), ('I', 3)]
}
```

So from **E**, you can go to:

- H (cost 2)
- I (cost 3)

🔹 Expansion Step

From:

```python
(5, ['S', 'A', 'E'])
```

We expand to:

👉 **For neighbor H:**

```python
new_cost = 5 + 2 = 7
new_path = ['S', 'A', 'E', 'H']
```

👉 **For neighbor I:**

```python
new_cost = 5 + 3 = 8
new_path = ['S', 'A', 'E', 'I']
```

These are added:

```python
candidates.append((7, ['S','A','E','H']))
candidates.append((8, ['S','A','E','I']))
```

🔹 What "expand" REALLY means

🔹 The line:

Python Run

```python
for neighbor, edge_cost in graph.get(current, []):
```

🔹 Why `[]` is used?

👉 `graph.get(current, [])` means:

> "Get neighbors of `current` node —
> **if it doesn't exist, return an empty list instead of crashing"**

Get Plus ×

🔹 Without `[]` (problem)

If you wrote:

Python Run

```python
graph[current]
```

And `current` is **not in the graph**, Python will give:

❌ **KeyError (program crashes)**

🔹 Line 8

python

```python
if not candidates:
    return None, float('inf')
```

```
👉 If no candidates generated — **dead end, no path exists**
```

```
candidates = []  → no neighbors found → return None ❌
float('inf')     → infinity, meaning no valid cost
```

Get Plus ×

🔍 Why can this happen?

- You reached nodes that have **no neighbors**
- All paths ended (dead ends)

🔥 Example:

You're at node `E`, but:

```python
graph['E'] = []
```

Run

👉 No expansion → `candidates = []`

So:

- No way forward
- Goal not found

👉 Return:

```python
(None, ∞)
```

Run

✔ `None` → no path found

✔ `inf` → cost is meaningless (failure)

🔹 2.

```python
candidates.sort(key=lambda x: x[0])
```

Run

✅ Meaning:

> "Sort all candidate paths based on cost"

🔥 Example:

Before sorting:

```python
[(7, ['S','A','E','H']),
 (5, ['S','B','F']),
 (9, ['S','C','G'])]
```

Run

After sorting:

```python
[(5, ...),
 (7, ...),
 (9, ...)]
```

Run

✔ Lowest cost first

✔ Best paths on top

🔹 2. `[:number]` → First elements (slicing)

This is what you're asking about 👇

Python

```
candidates[:beam_width]
```

✅ Meaning:

> "Take elements from start up to `beam_width` (not including it)"

🔥 Example:

Python

```
candidates = [A, B, C, D]
beam_width = 2
```

Python

```
candidates[:2] → [A, B]
```

- ✔ First 2 elements
- ✔ Rest ignored

🔹 General Rule of Slicing

Get Plus ×

Python

```
list[start : end]
```

- `start` = where to begin (default = 0)
- `end` = where to stop (NOT included)

✅ Common cases:

1.

Python

```
[:k]
```

👉 First `k` elements

2.

Python

```
[k:]
```

👉 From index `k` → till end

🔹 The line you're asking about:

Python Run

```python
return None, float('inf')
```

(at the very end of the function)

🔹 Short Answer

👉 It's a **fallback return**

> "If the loop ends and we STILL didn't find the goal → return failure"

```python
# BEAM SEARCH ALGORITHM
# This code implements Beam Search for finding the shortest path in a graph


graph = {
    'S': [('A', 3), ('B', 6), ('C', 5)],
    'A': [('D', 9), ('E', 8)],
    'B': [('F', 12), ('G', 14)],
    'C': [('H', 7)],
    'H': [('I', 5), ('J', 6)],
    'I': [('K', 10), ('L', 2)],
    'D': [], 'E': [], 'F': [], 'G': [], 'J': [], 'K': [], 'L': []
}
```

```python

def beam_search(start, goal, beam_width=2):
    beam = [(0, [start])]

    while beam:
        candidates = []

        for cost, path in beam:
            current = path[-1]

            if current == goal:
                return path, cost

            for neighbor, edge_cost in graph.get(current, []):
                new_cost = cost + edge_cost
                new_path = path + [neighbor]
                candidates.append((new_cost, new_path))
    #if there is no expanded nodes from current node ie candidates
        if not candidates:
            return None, float('inf')
    #there are candidates so sort it
        candidates.sort(key=lambda x: x[0])
        beam = candidates[:beam_width]

    return None, float('inf')
```

```python

# Run the code
start_node = 'S'
goal_node = 'L'
beam_width = 2

print(f"Beam Search from {start_node} to {goal_node}")
print(f"Beam Width: {beam_width}\n")

path, cost = beam_search(start_node, goal_node, beam_width)

if path:
    print(f"Path found: {' -> '.join(path)}")
    print(f"Total cost: {cost}")
else:
    print("No path found!")
```

HILL CLIMBING

🔹 What is Hill Climbing? (Core Idea)

> "Start somewhere and keep moving to a **better neighboring state** until no improvement is possible."

👉 It's a **local search algorithm**

- Doesn't care about path
- Only cares about **current state → better state**

- Only cares about **current state → better state**

🔹 Real-Life Intuition

Imagine you're climbing a hill in fog:

- You can only see nearby points
- You move where height increases
- You stop when no nearby point is higher

👉 That's Hill Climbing

🔹 General Code (we'll break this)

Python Run

```python
def hill_climbing(start):
    current = start

    while True:
        neighbors = get_neighbors(current)

        if not neighbors:
            return current

        next_state = best_neighbor(neighbors)

        if value(next_state) <= value(current):
            return current

        current = next_state
```

1.

```python
current = start
```

👉 You begin from some initial state

Example (N-Queens later):

- A random board arrangement

2.

```python
neighbors = get_neighbors(current)
```

👉 Generate **all possible small changes**

```python
if not neighbors:
    return current
```

👉 If no moves possible → stop

✔ Rare, but safe check
✔ Same idea as beam search safety

4.

```python
next_state = best_neighbor(neighbors)
```

👉 Pick the **best among all neighbors**

🔥 Important:

"Best" depends on a function:

```python
value(state)
```

Example:

```python
neighbors = [4, 6]
```

If:

```python
value(6) > value(4)
```

👉 Choose `6`

🔸 5. (MOST IMPORTANT LINE 🔥)

```python
if value(next_state) <= value(current):
    return current
```

👉 Meaning:

> "If no improvement → STOP"

Why?

Because:

- Hill Climbing ONLY moves **uphill (better states)**
- If no better neighbor exists → you are stuck

This is called:

> **Local Maximum**

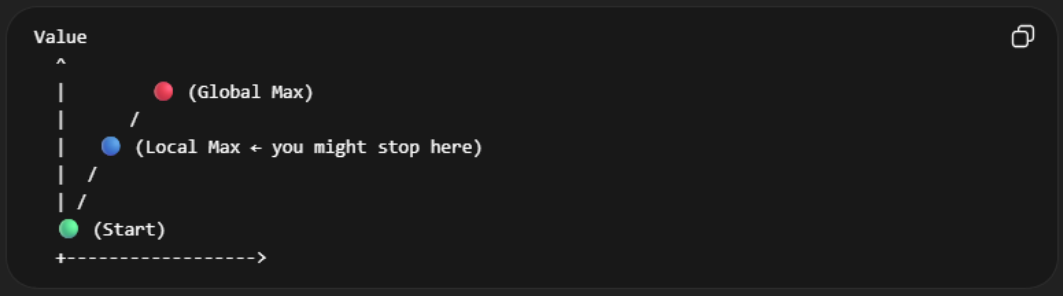
```python
current = next_state
```

👉 Move to better state and repeat

🔹 Full Flow in Simple Words

1. Start somewhere
2. Look at neighbors
3. Pick the best one
4. If better → move
5. If not → stop

🔹 Visual Example



👉 Hill climbing might stop at 🔵 instead of 🔴

🔹 Key Concepts YOU MUST KNOW

⚠️ 1. Local Maximum

- You stop at a point better than neighbors
- But not the best overall

⚠️ 2. Plateau

- All neighbors have same value

👉 Algorithm gets stuck

⚠️ 3. Ridge

- Path to better solution exists
- But requires sideways/down move

👉 Hill climbing fails

🔹 Why no path stored?

Unlike Beam Search:

🔹 Simple Analogy

- **Beam Search**: Try multiple paths
- **Hill Climbing**: Follow ONE path blindly

🔹 Why this helps for N-Queens

In N-Queens:

- `current` = board
- `neighbors` = all boards with one queen moved
- `value()` = number of conflicts (we try to MINIMIZE)

N QUEENS:

🔹 Why Count Attacking Queen Pairs?

In N-Queens:

- **Goal:** Place N queens on an N×N board so that **no two queens attack each other**.
- A queen attacks another if:
  1. They are in the **same column**
  2. They are in the **same row** (usually we avoid this by design)
  3. They are in the **same diagonal**

Step 1: Measure "How bad is the board?"

- Hill climbing is a **local search** algorithm.
- We need a **heuristic** to decide which state (board) is better.
- **Conflicts = badness measure:**
  - More conflicts → worse board
  - Fewer conflicts → better board
  - 0 conflicts → perfect solution

Step 2: Use Conflicts to Move

1. Start with a random board (may have conflicts).
2. Look at all possible "neighbor boards" (move one queen).
3. Count conflicts for each neighbor.
4. Pick the neighbor with **fewer conflicts** → move there.

> This is exactly the same as "climbing uphill" but in reverse: fewer conflicts = better.

Step 3: Stop When Optimal or Stuck

- If conflicts = 0 → we found a solution
- If **no neighbor has fewer conflicts** → local minimum → stuck

---

Counting conflicts gives the algorithm a **way to evaluate which board is better**.
Without this, hill climbing **would not know which move improves the solution**.

Simple Hill Climbing N-Queens — Full Code Walkthrough

🔹 Overall Code At A Glance

```
CREATE random board (index=row, value=col)
    ↓
COUNT attacking queen pairs (conflicts)
    ↓
GENERATE all neighbor boards
    ↓
Find FIRST neighbor that is better
    ↓
Move to it immediately (don't check rest)
    ↓
If no better neighbor → STUCK → stop
    ↓
Repeat until conflicts=0 or stuck
```

🔹 2. Where the "initial state" comes from

Look at this line in the code:

Python Run

```python
current_state = [random.randint(0, n - 1) for _ in range(n)]
```

- This generates a **random state** for the board
- For example, for n=8, you might get:

Python Run

```python
current_state = [2, 0, 6, 4, 3, 7, 1, 5]
```

- This is **the first state** that hill climbing starts from.

🔹 2. Applying it to N-Queens

```python
for i in range(n):
    for j in range(i+1, n):
        ...
```

- `i` goes from `0` to `n-1` → picks the **first queen**
- `j` goes from `i+1` to `n-1` → picks the **second queen** for comparison

Why `i+1`?

- To **skip the queen itself** (don't compare i with i)
- To **avoid double-counting pairs** (we already compared previous queens)

🔹 5. Quick Visual Trick

- Draw two queens on a grid
- Count how many rows apart = how many columns apart
- If **same** → diagonal

```
Q . . .        row difference = 1, col difference = 1 → diagonal
. Q . .
```

- If **different** → not diagonal

So, in short:

```python
abs(state[i]-state[j]) == abs(i-j)
```

exactly detects diagonal attacks ✅

🔹 1. What does it check?

- `state[i]` = column of queen in row `i`
- `state[j]` = column of queen in row `j`

So:

```python
state[i] == state[j]
```

checks if two queens are in the same column.

🔹 2. Why do we need this?

- In chess, queens attack vertically in the same column.
- If two queens are in the same column, they **can attack each other**, so it counts as a **conflict**.

3. Example

Suppose `state = [0, 0, 2, 3]` (n=4)

- Row 0 → Column 0
- Row 1 → Column 0

Check `i=0, j=1`:

Python Run

```
state[0] == state[1] → 0 == 0 → True
```

- ✅ Conflict!
- Two queens are in the **same column**

```python
def get_neighbors(state):
    neighbors = []
    n = len(state)
    for row in range(n):
        for col in range(n):
            if col != state[row]:
                new_state = list(state)
                new_state[row] = col
                neighbors.append(new_state)
    return neighbors
```

State is a list of queen position

State[i] here i is row and state[i] means we get column of queen from row i

State [2,0,3,5] these 2 , 0 , 3 , 5 are the column coordinates where queens are stationed

In state[1] . we have col 2 (queen at col 2) and and in state[2]we have column 0(queen at col 0) , so columns didn't match , so there is no conflict

Get Plus ×

2. Breaking down the code

Python Run

```
def get_neighbors(state):
    neighbors = []          # Step 1: start with an empty list
    n = len(state)          # Step 2: get the size of the board / number of queens
```

- `state` is a list of queen positions.
  Example: `state = [1, 3, 0, 2]` for 4-queens

Python Run

```
for row in range(n):
```

- Loop over each **row** (each queen).
- Row = 0 → first queen, row = 1 → second queen, etc.

Python Run

```
for row in range(n):
```

- Loop over each **row** (each queen).
- Row = 0 → first queen, row = 1 → second queen, etc.

Python Run

```
    for col in range(n):
        if col != state[row]:
```

- Loop over **all columns** in that row
- Skip the column where the queen **already is** (`col != state[row]`)

✅ This ensures we only move the queen to a **different column**, not stay in the same place.

Python Run

```
    for col in range(n):
        if col != state[row]:
```

- Loop over **all columns** in that row
- Skip the column where the queen **already is** (`col != state[row]`)

✅ This ensures we only move the queen to a **different column**, not stay in the same place.

Python Run

```
            new_state = list(state)
            new_state[row] = col
            neighbors.append(new_state)
```

Step by step:

1. `new_state = list(state)` → make a **copy** of the current state (so we don't modify the original)
2. `new_state[row] = col` → move the queen in the current row to a **new column**
3. `neighbors.append(new_state)` → add this **new board configuration** to the neighbors list

- In N-Queens:
    - A **neighbor** is a board that's almost the same as the current one, but you **move one queen to a different column**.

2. Why do we need a list of neighbors?

Python

Run

```
neighbors = []
```

- Because there are **multiple possible moves** from the current state:
  - Row 0 queen → maybe 3 different columns
  - Row 1 queen → maybe 3 different columns
  - ...etc.
- We **can't just pick one randomly**; hill climbing wants to **compare all neighbors** to see which is the best.
- So we store all of them in a **list**.

4. Analogy

- Imagine you're at a hilltop and can take **3 steps in front, left, right.**
- You write down all 3 positions → that's your **neighbors list**
- Then you check the **height** of each → choose the highest one.
- That's exactly what `neighbors` + `append` does in code.

---

5. Quick recap

Concept	Code Part	Why needed
Neighbor	`new_state`	A possible move from current board
List of neighbors	`neighbors = []`	Collect all possible moves
Append	`neighbors.append(new_state)`	Store each new candidate to compare later

1. Current state

Suppose we have a 4x4 board:

Python Run

```python
current_state = [1, 3, 0, 2]
# Row 0 → col 1
# Row 1 → col 3
# Row 2 → col 0
# Row 3 → col 2
```

---

2. Generating neighbors

For **each row**, move the queen to any **other column**.

Row 0 (queen at col 1) → can move to col 0, 2, 3:

Python Run

```python
[0, 3, 0, 2]
[2, 3, 0, 2]
[3, 3, 0, 2]
```

Row 1 (queen at col 3) → can move to col 0, 1, 2:

```python
[1, 0, 0, 2]
[1, 1, 0, 2]
[1, 2, 0, 2]
```

Row 2 (queen at col 0) → can move to col 1, 2, 3:

```python
[1, 3, 1, 2]
[1, 3, 2, 2]
[1, 3, 3, 2]
```

Row 3 (queen at col 2) → can move to col 0, 1, 3:

```python
[1, 3, 0, 0]
[1, 3, 0, 1]
[1, 3, 0, 3]
```


3. The `neighbors` list

So, after running `get_neighbors(current_state)`:

```python
neighbors = [
    [0, 3, 0, 2], [2, 3, 0, 2], [3, 3, 0, 2],  # row 0 moves
    [1, 0, 0, 2], [1, 1, 0, 2], [1, 2, 0, 2],  # row 1 moves
    [1, 3, 1, 2], [1, 3, 2, 2], [1, 3, 3, 2],  # row 2 moves
    [1, 3, 0, 0], [1, 3, 0, 1], [1, 3, 0, 3]   # row 3 moves
]
```

✅ Total of 12 neighbors (4 rows × 3 alternative columns each)

4. Picking a random neighbor

```python
import random

random_neighbor = random.choice(neighbors)
print("Random next move:", random_neighbor)
```

Example output:

```python
Random next move: [1, 3, 2, 2]
```

🔹 Line 4

```python
current_state = [random.randint(0, n-1) for _ in range(n)]
current_conflicts = calculate_conflicts(current_state)
```

👉 Creates random starting board and counts its conflicts

```
n = 8
current_state    = [3,6,2,7,1,4,0,5]  ← random
current_conflicts = 4                 ← 4 attacking pairs
```

🔹 Line 5

```python
while True:
    neighbors = get_neighbors(current_state)
    next_state = None
    next_conflicts = current_conflicts
```

👉 Loops forever until we break — generates all neighbors each iteration

```
next_state     = None      ← no better neighbor found yet
next_conflicts = 4         ← must beat this to move
```

🔹 Your code is using:

Get Plus ×

Python

```python
for neighbor in neighbors:
    neighbor_conflicts = calculate_conflicts(neighbor)

    if neighbor_conflicts < next_conflicts:
        next_state = neighbor
        next_conflicts = neighbor_conflicts
        break
```

🔹 What does this mean?

- It loops through neighbors one by one
- The **moment it finds a better one**, it stops (`break`)

👉 It does **NOT check all neighbors**

🔹 So correct logic is:

> "Pick the **first neighbor that improves the current state**, not necessarily the best one."

🔹 If you wanted the BEST neighbor

Then you would do:

Python

```python
best_neighbor, best_conflicts = min(neighbors, key=lambda x: x[1])
```

👉 This checks **ALL neighbors**

🔹 Line 7

```python
if next_conflicts >= current_conflicts:
    break
```

👉 If no neighbor was better — **stuck at local minimum, stop**

```
next_conflicts = 4  (never updated)
current_conflicts = 4
4 >= 4 → STUCK → break ❌

next_conflicts = 3  (was updated)
3 >= 4 → False → keep going ✅
```

```
71                  break
72
73              # Move to the better neighbor
74              current_state = next_state
75              current_conflicts = next_conflicts
76
77          return current_state, current_conflicts
78
79
80      # ========================
```

```
    # ========================
    # Run Hill Climbing
    # ========================
    n = 8
    solution, conflicts = simple_hill_climbing(n)

    if conflicts == 0:
        print("Solution found:", solution)
    else:
        print("Stuck with conflicts:", conflicts)
        print("State:", solution)


    # ========================
    # Explanation
    # ========================
    """
    Part 1: Why do we count conflicts?
```

8 PUZZLE

8-Puzzle Hill Climbing — Full Code Walkthrough

🔹 What is 8-Puzzle First

```
A 3x3 grid with 8 numbered tiles and 1 empty space (0)

Initial (random):          Goal:
+---+---+---+              +---+---+---+
| 5 | 3 |   |              | 1 | 2 | 3 |
+---+---+---+              +---+---+---+
| 2 | 8 | 6 |              | 4 | 5 | 6 |
+---+---+---+              +---+---+---+
| 1 | 7 | 4 |              | 7 | 8 |   |
+---+---+---+              +---+---+---+

Goal = arrange all tiles in order by sliding them into empty space
```

🔹 What are we doing on it

```
We apply HILL CLIMBING to solve it

Instead of:
  neighbors = move one queen to different row

Here:
  neighbors = slide empty space Up/Down/Left/Right

Instead of:
  conflicts = attacking queen pairs

Here:
  heuristic = number of misplaced tiles
```

🔹 Overall Code At A Glance

```
CREATE random 3x3 board
    ↓
CHECK if solvable (inversion count)
    ↓
COUNT misplaced tiles (heuristic)
    ↓
GENERATE neighbors (slide blank 4 directions)
    ↓
PICK neighbor with fewest misplaced tiles
    ↓
If better → move to it
If not better → STUCK → retry
    ↓
Repeat up to 10 attempts
```

Key difference from N-Queens Hill Climbing:

N-Queens HC	8-Puzzle HC
1D list — queen positions	2D grid — tile positions
Move queen to different row	Slide blank tile
conflicts = attacking pairs	heuristic = misplaced tiles
No solvability check	✅ checks if solvable first
Simple list copy	`copy.deepcopy` for 2D grid

🔹 Line 1

```python
GOAL_STATE = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]
```

👉 The target configuration — **0 is empty space at bottom right**

```
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 |   |
```

🔹 Line 2

```python
def create_random_initial_state():
    numbers = list(range(9))
    random.shuffle(numbers)
    state = []
    for i in range(3):
        row = numbers[i*3:(i+1)*3]
        state.append(row)
    return state
```

👉 Creates random starting board by **shuffling 0-8 into 3x3 grid**

```
numbers = [0,1,2,3,4,5,6,7,8]
shuffle → [5,3,0,2,8,6,1,7,4]

i=0 → row = numbers[0:3] = [5,3,0]
i=1 → row = numbers[3:6] = [2,8,6]
i=2 → row = numbers[6:9] = [1,7,4]

state = [[5,3,0],
         [2,8,6],
         [1,7,4]]
```

🔹 Step 1: Flatten the Grid (ignore 0)

Python Run

```python
flat = []
for row in state:
    for val in row:
        if val != 0:
            flat.append(val)
```

👉 This converts the 2D grid into a 1D list **without 0**

So:

```
Original:
[5, 3, 0
 2, 8, 6
 1, 7, 4]

Flattened (ignoring 0):
flat = [5, 3, 2, 8, 6, 1, 7, 4]
```

🔹 Step 2: Count Inversions

Python Run

```python
for i in range(len(flat)):
    for j in range(i+1, len(flat)):
        if flat[i] > flat[j]:
            inversions += 1
```

👉 An **inversion** means:

> A bigger number appears **before** a smaller number

👉 Checks if puzzle **can actually be solved** before trying
```
state = [[5,3,0],
         [2,8,6],
         [1,7,4]]

flat = [5,3,2,8,6,1,7,4]  ← remove 0

inversions = count pairs where left > right:
5>3 ✅  5>2 ✅  5>1 ✅  5>4 ✅
3>2 ✅  3>1 ✅
8>6 ✅  8>1 ✅  8>7 ✅  8>4 ✅
6>1 ✅  6>4 ✅
7>4 ✅
total = 13 inversions

13 % 2 = 1 → ODD → NOT solvable ❌
```

> Not all shuffled puzzles are solvable **Even inversions = solvable, Odd inversions = impossible** Saves time by not attempting impossible puzzles

---

🔹 Line 4

python

```python
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j
```

👉 Finds **row and column of empty space (0)**

```
state = [[5,3,0],
         [2,8,6],
         [1,7,4]]

i=0, j=2 → state[0][2] = 0 → return (0,2)
blank is at row 0, col 2
```

🔹 Line 5

```python
def calculate_heuristic(state):
    misplaced = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != 0 and state[i][j] != GOAL_STATE[i][j]:
                misplaced += 1
    return misplaced
```

👉 Counts tiles **not in their goal position**

```
current:          goal:
| 5 | 3 |   |     | 1 | 2 | 3 |
| 2 | 8 | 6 |     | 4 | 5 | 6 |
| 1 | 7 | 4 |     | 7 | 8 |   |

position (0,0): 5 != 1 → misplaced ❌
position (0,1): 3 != 2 → misplaced ❌
position (0,2): 0 → skip
position (1,0): 2 != 4 → misplaced ❌
position (1,1): 8 != 5 → misplaced ❌
position (1,2): 6 == 6 → correct ✅
position (2,0): 1 != 7 → misplaced ❌
position (2,1): 7 != 8 → misplaced ❌
position (2,2): 4 != 0 → misplaced ❌
```

👉 Counts tiles **not in their goal position**

```
current:          goal:
| 5 | 3 |   |     | 1 | 2 | 3 |
| 2 | 8 | 6 |     | 4 | 5 | 6 |
| 1 | 7 | 4 |     | 7 | 8 |   |

position (0,0): 5 != 1 → misplaced ❌
position (0,1): 3 != 2 → misplaced ❌
position (0,2): 0 → skip
position (1,0): 2 != 4 → misplaced ❌
position (1,1): 8 != 5 → misplaced ❌
position (1,2): 6 == 6 → correct ✅
position (2,0): 1 != 7 → misplaced ❌
position (2,1): 7 != 8 → misplaced ❌
position (2,2): 4 != 0 → misplaced ❌

misplaced = 7
```

🔹 **Line 6** ⭐

python

```python
def get_neighbors(state):
    neighbors = []
    blank_row, blank_col = find_blank(state)
    moves = [(-1,0),(1,0),(0,-1),(0,1)]  # Up Down Left Right

    for dr, dc in moves:
        new_row = blank_row + dr
        new_col = blank_col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_state = copy.deepcopy(state)
            new_state[blank_row][blank_col] = new_state[new_row][new_col]
            new_state[new_row][new_col] = 0
            neighbors.append(new_state)
    return neighbors
```

👉 Generates all states reachable by **sliding blank in 4 directions**

```
So 2 neighbors generated

Move Down (swap blank with tile below):
before:  [[5,3,0],[2,8,6],[1,7,4]]
after:   [[5,3,6],[2,8,0],[1,7,4]]  ← 6 moved up, blank moved down

Move Left (swap blank with tile to left):
before:  [[5,3,0],[2,8,6],[1,7,4]]
after:   [[5,0,3],[2,8,6],[1,7,4]]  ← 3 moved right, blank moved left
```

> `copy.deepcopy` needed because 2D list — normal copy would change original

python

```
best_neighbor = None
best_h = current_h

for neighbor in neighbors:
    h = calculate_heuristic(neighbor)
    if h < best_h:
        best_h = h
        best_neighbor = neighbor

if best_neighbor is None:
    return current, moves, False

current = best_neighbor
current_h = best_h
moves += 1
```

👉 Finds best neighbor — **steepest ascent style** (checks all, picks best)

🔹 Big Idea First

You are standing at a **current state** and you look at all **neighbors (possible moves)**.

👉 You choose the one that has the **lowest heuristic value (h)**
(smaller h = closer to goal)

🔹 Line-by-Line Explanation

1. Initialize best choice

Python — Run

```
best_neighbor = None
best_h = current_h
```

- `best_neighbor = None` → we haven't chosen any move yet
- `best_h = current_h` → assume current state is best for now

2. Check all neighbors

Python Run

```
for neighbor in neighbors:
    h = calculate_heuristic(neighbor)
```

👉 For each possible move:

- Compute its **heuristic value (h)**
- This tells how far it is from goal

---

3. Compare and pick better one

Python Run

```
if h < best_h:
    best_h = h
    best_neighbor = neighbor
```

👉 If this neighbor is **better (smaller h)** than current:

- Update best value
- Store that neighbor

💡 Think:

> "Oh this move looks closer to goal than where I am now"

---

5. Move to best neighbor

Python Run

```
current = best_neighbor
current_h = best_h
moves += 1
```

👉 If better move exists:

- Go there (update current)
- Update heuristic
- Increase move count

🔹 **Final Understanding**

This code is basically doing:

"From all possible moves, pick the one that improves the situation.
If no improvement is possible → stop."

🔹 Line 8

python

```
def print_state(state, title="State"):
    print(f"\n{title}:")
    print("-" * 13)
    for row in state:
        print("|", end="")
        for val in row:
            if val == 0:
                print("   |", end="")
            else:
                print(f" {val} |", end="")
        print()
        print("-" * 13)
```

👉 Prints the board in a **readable grid format**

```
State:
-------------
| 5 | 3 |   |
-------------
| 2 | 8 | 6 |
-------------
| 1 | 7 | 4 |
-------------
```

```
# ============================================================
# 8-PUZZLE HILL CLIMBING — HOW IT WORKS
# ============================================================

# STEP 1 — SETUP
# Create random 3x3 board → shuffle 0-8 into grid
# Check solvability → even inversions = solvable ✅

# STEP 2 — HEURISTIC
# Count misplaced tiles vs goal state
# 0 = goal reached, higher = further from goal

# STEP 3 — GENERATE NEIGHBORS
# Find blank tile (0) position
# Try sliding blank Up/Down/Left/Right
# Each valid slide = one neighbor state

# STEP 4 — PICK BEST NEIGHBOR
# Check heuristic of all neighbors
# Pick one with lowest misplaced tiles
# If none better → STUCK at local minimum ❌

# STEP 5 — MOVE OR STOP
# best_h < current_h → move to best neighbor ✅
# best_neighbor is None → stuck → return False

# STEP 6 — RETRY
# Try up to 10 random starting states
# Different start may avoid local minimum

# ============================================================
```

```python
# this part IS the main
max_attempts = 10
solved = False

for attempt in range(1, max_attempts + 1):
    initial = create_random_initial_state()

    if not is_solvable(initial):
        print("Puzzle not solvable")
        continue

    final, moves, success = hill_climbing(initial)

    if success:
        print(f"Solved in {moves} moves")
        solved = True
        break
    else:
        print(f"Stuck after {moves} moves")
```

🔹 What it does step by step

```
repeat up to 10 times:

  STEP 1 → create random board
  STEP 2 → check if solvable → if not skip this attempt
  STEP 3 → run hill climbing
  STEP 4 → if solved → print + stop ✅
           if stuck  → try again ❌
```

GENETIC ALGORITHM:

How to think about this (VERY IMPORTANT)

Hill Climbing:

```
ONE state → improve step by step
```

Genetic Algorithm:

```
MANY states → select → mix → mutate → repeat
```

Final intuition

- `population` = many boards
- `select_parent()` = pick good boards
- `crossover()` = combine them
- `mutate()` = random changes
- Loop = evolution

---

1. State Representation (same as before)

Python Run

```python
state = [1, 3, 0, 2, ...]
```

- Index = row
- Value = column

✅ Same as hill climbing

---

2. Fitness Function

Python Run

```python
def calculate_conflicts(state):
```

👉 Counts number of attacking queen pairs

- Same column → conflict
- Same diagonal → conflict

✔ Lower = better
✔ 0 = perfect solution

3. Create Initial Population

Python Run

```python
population = create_population(n, population_size)
```

👉 Instead of ONE state:

```
population = [
  [1,3,0,2,...],
  [2,0,3,1,...],
  [0,4,7,5,...],
  ...
]
```

✔ Many random boards

Get Plus ×

4. Main Loop (Generations)

Python Run

```python
for generation in range(max_generations):
```

👉 This is like:

```
Generation 0 → Generation 1 → Generation 2 → ...
```

Each loop = **evolution step**

---

5. Find Best Solution

Python Run

```python
best = min(population, key=calculate_conflicts)
best_conflicts = calculate_conflicts(best)
```

👉 From all boards:

- Pick the one with **least conflicts**

Genetic Algorithm N-Queens — Full Code Walkthrough

🔹 Overall Code At A Glance

```
CREATE population of 50 random boards
    ↓
CHECK best board in population
    ↓
SELECT parents via tournament (pick 3, best wins)
    ↓
CROSSOVER — mix two parents to make child
    ↓
MUTATE — randomly change some genes
    ↓
REPEAT for 500 generations
    ↓
Return best solution found
```

Key difference from Hill Climbing:

Hill Climbing	Genetic Algorithm
One board at a time	Population of 50 boards
Move to better neighbor	Breed new boards from parents
Gets stuck at local min	Crossover escapes local minima
No generations	500 generations
One solution evolves	Entire population evolves

🔹 Core Concepts First

```
Individual  = one board  = [3,6,2,7,1,4,0,5]
Population  = 50 boards  = [ind1, ind2, ... ind50]
Generation  = one full cycle of select+crossover+mutate
Fitness     = conflicts  = lower is better
Gene        = one queen position in the board
```

🔹 Line 1

```python
def calculate_conflicts(state):
    conflicts = 0
    n = len(state)
    for i in range(n):
        for j in range(i+1, n):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                conflicts += 1
    return conflicts
```

```
👉 Same as Hill Climbing — counts **attacking queen pairs**
```

```
state = [3,6,2,7,1,4,0,5]
conflicts = 1   ← only 1 attacking pair
```

> In Genetic Algorithm this is called the **fitness function** Lower conflicts = higher fitness = better individual

🔹 5. Final Output Example

If:

```python
n = 4
size = 3
```

Then:

```python
population = [
    [2,0,3,1],
    [1,3,0,2],
    [0,2,1,3]
]
```

👉 A list of 3 different boards

🔹 6. Summary

Part	Meaning
`_ in range(size)`	create `size` individuals
`_ in range(n)`	each individual has `n` queens
`_`	loop variable not needed

🔹 Line 2

```python
def create_population(n, size):
    population = []
    for _ in range(size):
        individual = [random.randint(0, n-1) for _ in range(n)]
        population.append(individual)
    return population
```

👉 Creates **50 random boards** to start with

```
n=8, size=50

individual = [random board]
population = [
    [3,6,2,7,1,4,0,5],   ← individual 1
    [1,4,6,3,0,7,2,5],   ← individual 2
    [2,0,6,4,7,1,3,5],   ← individual 3
    ...50 total
]
```

> Hill Climbing started with **1 random board** ↓ …etic Algorithm starts with **50 random boards** — much better coverage

After creating population , select parents

🔹 Line 3 ⭐ (New Concept — Tournament Selection)

```python
def select_parent(population):
    sample = random.sample(population, 3)
    best = min(sample, key=calculate_conflicts)
    return best
```

👉 Picks **3 random individuals** from population — returns the best one

```
population has 50 individuals

random.sample picks 3:
  [3,6,2,7,1,4,0,5]  conflicts=1
  [1,1,1,1,1,1,1,1]  conflicts=28
  [2,4,6,0,3,1,7,5]  conflicts=2

min by conflicts → [3,6,2,7,1,4,0,5]  ← winner, conflicts=1
```

> Like a **tournament** — 3 compete, best becomes parent Better individuals more likely to be selected but not guaranteed This keeps **diversity** in population

1. What is the goal here?

Get Plus ×

👉 We want to **choose a good parent** from the population

But NOT always the absolute best (to keep diversity)

---

2. Step 1: Random Sampling

Python

Run

```python
sample = random.sample(population, 3)
```

👉 This means:

> "Pick 3 random individuals from the population"

Example:

Python

Ru

```python
population = [
  [1,3,0,2],
  [2,0,3,1],
  [0,2,1,3],
  [3,1,2,0],
  [1,0,2,3]
]
```

Sample might be:

Python

Ru

```python
sample = [
  [2,0,3,1],
  [3,1,2,0],
  [1,0,2,3]
]
```

✔ Random selection

🔹 3. Step 2: Pick the Best from Sample

Python

```python
best = min(sample, key=calculate_conflicts)
```

👉 This means:

> "From these 3, pick the one with the least conflicts"

Example:

```
[2,0,3,1] → 2 conflicts
[3,1,2,0] → 4 conflicts
[1,0,2,3] → 1 conflict  ✅ BEST
```

👉 So:

Python

```python
best = [1,0,2,3]
```

🔹 Your doubt

> "We only select one best parent, so where do `parent1` and `parent2` come from?"

🔹 🔥 Key Idea

👉 We call the selection function **TWICE**

🔹 Look at this part in your GA:

Python Run

```python
parent1 = select_parent(population)
parent2 = select_parent(population)
```

👉 This means:

- First call → gives **parent1**
- Second call → gives **parent2**

Step 3: Create new population

Python Run

```python
new_population = []

while len(new_population) < population_size:
    parent1 = select_parent(population)
    parent2 = select_parent(population)

    child = crossover(parent1, parent2)
    child = mutate(child, mutation_rate)

    new_population.append(child)
```

Let's break this down:

1. **Select parents**
   - `select_parent(population)` → randomly chooses a few boards (tournament) and picks the one with **least conflicts**.
   - We do this **twice** to get `parent1` and `parent2`.
2. **Crossover**

Python Run

```python
child = crossover(parent1, parent2)
```

   - Combines parts of both parents to create a **new board (child)**.
   - Think of it as **mixing genes**: some queens from parent1, some from parent2.

```python
child = mutate(child, mutation_rate)

new_population.append(child)
```

Let's break this down:

1. **Select parents**
   - `select_parent(population)` → randomly chooses a few boards (tournament) and picks the one with **least conflicts**.
   - We do this **twice** to get `parent1` and `parent2`.
2. **Crossover**

```python
child = crossover(parent1, parent2)
```

   - Combines parts of both parents to create a **new board (child)**.
   - Think of it as **mixing genes**: some queens from parent1, some from parent2.
3. **Mutation**

```python
child = mutate(child, mutation_rate)
```

   - Randomly changes some queens in the child **based on mutation_rate**.
   - Adds diversity to avoid getting stuck in a "bad configuration".
4. **Add child to new population**
   - Repeat until we have `population_size` new boards.

Ah! Let's focus on this line:

```python
while len(new_population) < population_size:
```

What it means:

- `new_population` → starts as an **empty list**: `[]`.
- `population_size` → total number of boards we want in **each generation**. For example, 50.

Step by step:

1. **Check the length of** `new_population`
   - `len(new_population)` → how many children we have already created.
2. **Loop while it is less than** `population_size`
   - Keep generating new children (via **selection** → **crossover** → **mutation**) until we have exactly `population_size` boards.
3. **Once we reach** `population_size`
   - The loop stops.
   - Now we have a full new generation ready to replace the old one.

````
python

new_population = []
while len(new_population) < population_size:
    parent1 = select_parent(population)
    parent2 = select_parent(population)
    child = crossover(parent1, parent2)
    child = mutate(child, mutation_rate)
    new_population.append(child)
population = new_population
```
👉 Builds entire new population of 50 children
```
repeat until 50 children created:

  parent1 = tournament(population)  ← best of 3 random
  parent2 = tournament(population)  ← best of 3 random

  child = crossover(p1, p2)         ← mix at random point
  child = mutate(child, 0.1)        ← randomly change 10%

  new_population.append(child)

old population COMPLETELY replaced by new population
```
> Unlike Hill Climbing which kept one board
> Here **entire population replaced** every generation
````

````
---

## 🔹 Full Generation Trace
```
Generation 0:
  population = 50 random boards
  best conflicts = 5

Generation 1:
  select parents → crossover → mutate → 50 new children
  best conflicts = 4  ← improved!

Generation 2:
  select parents → crossover → mutate → 50 new children
  best conflicts = 3

...

Generation 47:
  best conflicts = 0 → SOLUTION FOUND 🎯
  return [4,6,0,2,7,1,3,5]
````

The Code

Python

```python
def crossover(parent1, parent2):
    n = len(parent1)
    point = random.randint(1, n-1)
    child = parent1[:point] + parent2[point:]
    return child
```

1. Inputs

- `parent1` → a list representing one solution (like `[1, 3, 0, 2]`)
- `parent2` → another solution (like `[2, 0, 3, 1]`)

> Both parents are from the **population** and usually **good solutions**.

2. Length of the parent

Python

```python
n = len(parent1)
```

- `n` is the size of the N-Queens board (e.g., 8)
- It tells us **how many positions** we have to consid

3. Choose a crossover point

Python

```python
point = random.randint(1, n-1)
```

- Pick a **random index** where we will "split" the parents
- Example: if `n=4`, `point` could be `2`

> This ensures we take part from parent1 and part from parent2.

4. Combine parents

Python Run

```python
child = parent1[:point] + parent2[point:]
```

- `parent1[:point]` → take **first part** from parent1 (up to `point-1`)
- `parent2[point:]` → take **remaining part** from parent2 (from `point` to end)

Example

Python Run

```python
parent1 = [1, 3, 0, 2]
parent2 = [2, 0, 3, 1]
point = 2
```

- `parent1[:2]` → `[1, 3]`
- `parent2[2:]` → `[3, 1]`

Combine:

Python Run

```python
child = [1, 3] + [3, 1] = [1, 3, 3, 1]
```

> ✅ Child is a **new solution** combining traits of both parents.

````
👉 **Mixes two parents** at a random point to create one child
```
parent1 = [3, 6, 2, 7, 1, 4, 0, 5]
parent2 = [1, 4, 6, 0, 3, 1, 7, 5]

random point = 3

child = parent1[:3] + parent2[3:]
      = [3, 6, 2]   + [0, 3, 1, 7, 5]
      = [3, 6, 2, 0, 3, 1, 7, 5]
```
```
Visual:
parent1: [3, 6, 2, | 7, 1, 4, 0, 5]
parent2: [1, 4, 6, | 0, 3, 1, 7, 5]
                   ↑
              split here (point=3)

child:   [3, 6, 2, | 0, 3, 1, 7, 5]
          ←from p1→  ←——from p2——→
````
````

The Code

Python

```python
def mutate(state, rate):
    n = len(state)
    new_state = list(state)
    for i in range(n):
        if random.random() < rate:
            new_state[i] = random.randint(0, n-1)
    return new_state
```

1. Inputs

- `state` → a list representing a board configuration (like `[1, 3, 0, 2]`)
- `rate` → mutation probability (e.g., 0.1 means 10% chance to mutate each queen)

> Mutation randomly changes positions of queens to explore new solutions.

2. Copy the state

Python

```python
new_state = list(state)
```

- Make a **copy** so that the original parent is not modified
- We will mutate `new_state` to create a new candidate

3. Loop through each queen

Python

```python
for i in range(n):
```

- `i` is the **row index**
- For each queen, we check if it should **mutate**

4. Decide if mutation happens

Python

```python
if random.random() < rate:
```

- `random.random()` → returns a number between 0 and 1
- If this number is **less than the mutation rate**, we mutate this queen

> Example: rate = 0.1 → ~10% chance a queen will be moved

2 Why use `random.random() < rate`

Python

```
if random.random() < rate:
```

- `random.random()` generates a number between 0 and 1.
- `rate` is a **small probability** (like 0.1 for 10%).
- The condition checks: "Do we **mutate this queen**?"
  - If the random number is smaller than 0.1 → yes, mutate
  - Otherwise → leave it as is

✅ This ensures **each queen has only a small chance to be mutated**, keeping most of the population intact while still exploring new possibilities.

3 Analogy

Think of it like **genetic variation in nature**:

- Most offspring inherit traits **unchanged** from their parents.
- Occasionally, a **mutation happens** → could be good or bad.
- This small random change allows evolution to **find better solutions** over generations.

So in short:

> The `if random.random() < rate:` condition ensures that **mutation happens randomly with a controlled probability**, keeping GA effective.

python

```python
def mutate(state, rate):
    n = len(state)
    new_state = list(state)
    for i in range(n):
        if random.random() < rate:
            new_state[i] = random.randint(0, n-1)
    return new_state
```

🔹 🔹 Example

Python

```
state = [1, 3, 0, 2]
rate = 0.5
```

- Loop through each row:
  1. `i = 0` → random < 0.5? No → stays 1
  2. `i = 1` → random < 0.5? Yes → new random column = 2 → `[1, 2, 0, 2]`
  3. `i = 2` → random < 0.5? No → stays 0
  4. `i = 3` → random < 0.5? No → stays 2
- Final mutated state: `[1, 2, 0, 2]`

> ✅ Only row 1 got mutated randomly

🔹 Why mutation is important

1. Prevents **population from getting stuck** in local minima
2. Introduces **diversity** in the solutions
3. Helps GA explore **new configurations** for N-Queens

🔹 Line 6 ⭐ (Main Algorithm)

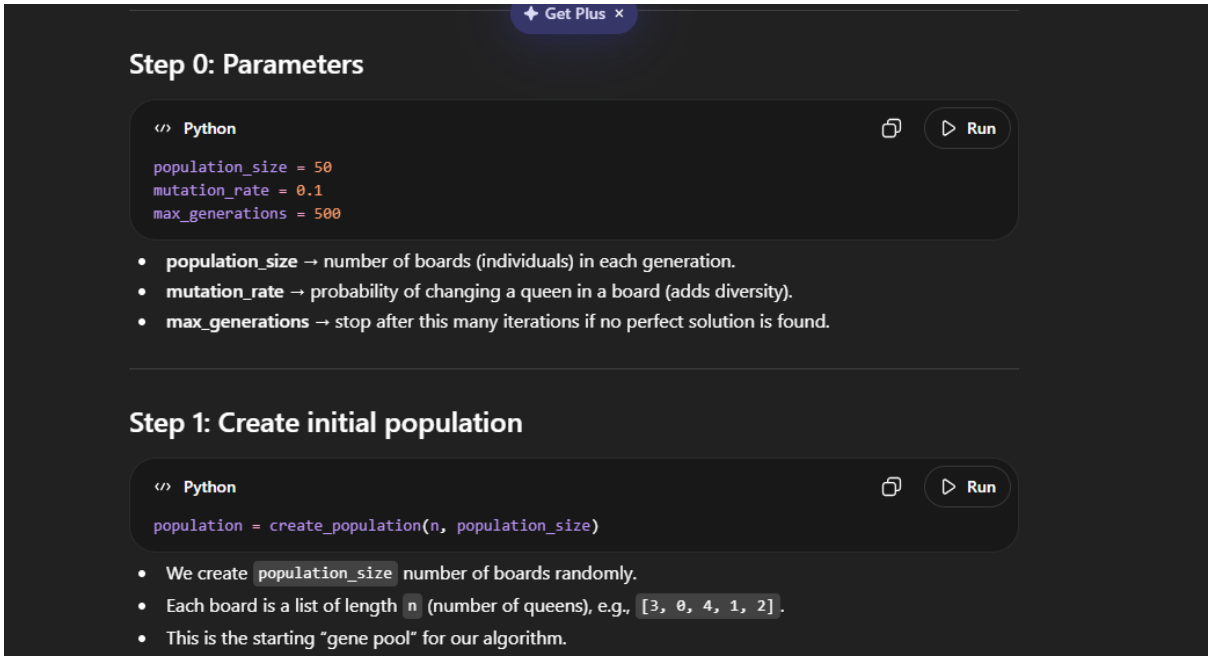
python

```
def genetic_algorithm(n):
    population_size = 50
    mutation_rate = 0.1
    max_generations = 500
    population = create_population(n, population_size)
```

```
👉 Sets up parameters and creates initial population
```

```
population_size  = 50   ← 50 boards at all times
mutation_rate    = 0.1  ← 10% chance each gene mutates
max_generations  = 500  ← try for 500 generations max
```

GENETIC FUNCTION


Get Plus ×
Step 0: Parameters
Python
Run
population_size = 50
mutation_rate = 0.1
max_generations = 500
population_size → number of boards (individuals) in each generation.
mutation_rate → probability of changing a queen in a board (adds diversity).
max_generations → stop after this many iterations if no perfect solution is found.
Step 1: Create initial population
Python
Run
population = create_population(n, population_size)
We create population_size number of boards randomly.
Each board is a list of length n (number of queens), e.g., [3, 0, 4, 1, 2].
This is the starting "gene pool" for our algorithm.


Step 2: Evaluate population and check best solution
Python
Run
best = min(population, key=calculate_conflicts)
best_conflicts = calculate_conflicts(best)
if best_conflicts == 0:
print("Solution found at generation", generation)
return best, best_conflicts
min(population, key=calculate_conflicts) → finds the board with least conflicts.
If best_conflicts is 0 → perfect solution! → we stop and return it.
This is like nature picking the fittest individual.

SVM

🔹 Support Vector Machines (SVM)

Definition:

Support Vector Machines (SVM) are **supervised learning algorithms** used mainly for:

- Classification ✅
- Regression ✅

👉 Main idea:

Find the **best separating boundary (hyperplane)** between classes.

🔥 Core Concept

👉 SVM tries to:

"maximize the margin between two classes"

- **Margin** = distance between hyperplane and closest data points
- Closest points are called:
  👉 **Support Vectors** (VERY important term)

🔹 Types of SVM

1. Linear SVM

👉 Used when data is **linearly separable**

- A straight line (2D) or plane (higher dimension) separates classes
- No transformation needed

✔ Example:

- Two classes clearly divided by a straight line

---

2. Non-Linear SVM

👉 Used when data is **NOT linearly separable**

- Data is mapped to **higher dimension**
- Then a linear boundary is found there

✔ Trick used:

👉 **Kernel Trick**

🔥 Kernel Trick (VERY IMPORTANT)

👉 Instead of manually converting data to higher dimension,

SVM uses a **kernel function** to do it efficiently.

👉 *"Allows non-linear separation without explicitly transforming data"*

🔹 Simple intuition

👉 Think like this:

- In **2D (x, y)** → hyperplane = **line**
- In **3D (x, y, z)** → hyperplane = **plane**
- In higher dimensions → called **hyperplane**

🔹 In SVM context

👉 Hyperplane is the line/plane that:

- ✔ separates two classes
- ✔ is placed in the **best possible position**
- ✔ maximizes the **margin**

🔥 Example (very clear)

Suppose you have:

```
Class 1: • • •
Class 2: × × ×
```

A hyperplane could be:

```
• • •  |  × × ×
       ↑
   hyperplane
```

🔹 Mathematical form (simple)

```
w · x + b = 0
```

Where:

- `w` = weights
- `x` = input
- `b` = bias

👉

🔥 Simple idea

Sometimes data looks like this in 2D:

```
• • • •
  × × × ×
• • • •
```

👉 Try drawing a straight line... impossible ❌

🔹 So what SVM does

It says:

👉 "Let's move this data into a higher dimension where it *becomes separable*."

🔹 Why not just any line?

You can draw many separating lines:

```
Option 1: too close to points ❌
Option 2: balanced ✓
Option 3: again too close ❌
```

🔥 Problem with small margin

If hyperplane is too close:

- small noise changes classification
- model becomes unstable
- poor generalization

🔹 Why large margin is better

Maximizing margin means:

- ✔ more stable boundary
- ✔ better generalization on new data
- ✔ less overfitting

> Analogy: sklearn is like a kitchen appliance. You don't need to know how a blender motor works — you just put ingredients in and press blend. Same here: you give sklearn data and it "blends" it into a trained model.

The four imports in this code

```
from sklearn import datasets
```

Data sklearn comes with built-in sample datasets for practice. This imports the datasets module so we can load one.

```
from sklearn.svm import SVC
```

Algorithm SVC = Support Vector Classifier. This is the actual SVM algorithm we will train.

```
from sklearn.model_selection import train_test_split
```

Splitting A function that automatically splits your data into a training portion and a testing portion.

```
from sklearn.metrics import accuracy_score
```

Evaluation A function that compares predicted labels vs actual labels and returns how often the model was correct.

🔹 1. What is the dataset?

Python Run

```
X = iris.data
```

👉 This is your **input data (features)**:

- 150 rows (flowers)
- 4 columns (measurements)

Example columns:

- sepal length
- sepal width
- petal length
- petal width

So each flower = a **4D point**

```python
y = iris.target
```

👉 This is your **labels (answers)**:

- 0 → Setosa
- 1 → Versicolor
- 2 → Virginica

So originally:
👉 This is a **3-class problem**

🔥 2. Why do we change labels?

```python
y = (y == 0).astype(int)
```

This is the important part.

🔥 2. Why do we change labels?

```python
y = (y == 0).astype(int)
```

This is the important part.

🔹 What this line does

It converts:

Original y	Meaning	New y
0	Setosa	1
1	Not Setosa	0
2	Not Setosa	0

🔥 So now the problem becomes:

👉 Instead of 3 classes → we now have **2 classes**

New meaning:

```
1 → Is Setosa ✓
0 → Not Setosa ✗
```

🔹 Why do we do this?

Because **SVM in your example is binary classifier**

👉 It can only answer:

- YES / NO
- Class A / Class B

So we simplify the problem.

🔹 One-line exam answer

👉 *This line converts a multi-class classification problem into a binary classification problem by labeling Setosa as 1 and all other classes as 0, allowing SVM to perform a simple yes/no classification task.*

Every line explained

Step 1 — Load data

```
iris = datasets.load_iris() X = iris.data y = iris.target y = (y == 0).astype(int)
```

Load `load_iris()` returns an object with the flower data already inside it.

`iris.data` = the measurements (4 numbers per flower). Stored in X (capital X = inputs/features).

`iris.target` = the correct answers (which flower). Stored in y (lowercase y = output/label).

Step 2 — Split data into train and test

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.3, random_state=42 )
```

Split This one function call creates 4 variables at once:

- `X_train` — 70% of flower measurements → model learns from these
- `X_test` — 30% of flower measurements → used only for testing
- `y_train` — correct answers for training rows
- `y_test` — correct answers for test rows (hidden from model during training)

`test_size=0.3` → reserve 30% for testing. `random_state=42` → fixed seed so the split is the same every time you run it.

```
svm = SVC(kernel='rbf', C=1, gamma='scale') svm.fit(X_train, y_train)
```

Train `SVC(...)` creates the model object with settings (parameters). `.fit()` is where the actual learning happens — the model studies X_train and y_train to find patterns.

Parameters:

- `kernel='rbf'` — the math shape used to separate classes (RBF handles curved boundaries)
- `C=1` — how strict the model is; higher = tries harder to classify every point correctly
- `gamma='scale'` — controls how far each point's influence reaches; 'scale' = auto-calculated

Step 4 — Make predictions

```
y_pred = svm.predict(X_test)
```

Predict Feed the test flowers (measurements only, no labels) into the trained model. It returns its guesses: a list of 0s and 1s for each test flower.

Step 5 — Measure accuracy

```
print("SVM Accuracy:", accuracy_score(y_test, y_pred))
```

Evaluate Compare `y_pred` (model's guesses) against `y_test` (real answers). Returns a number between 0 and 1. E.g. 0.978 means the model was correct 97.8% of the time.

Quick cheat sheet

Term	Plain English
X	Inputs — the measurements you give the model
y	Outputs — the correct answer labels
X_train / y_train	Data the model learns from
X_test / y_test	Data used only to test — model never sees y_test during training
.fit()	Train the model
.predict()	Ask the model to guess labels for new data
accuracy_score	% of guesses that were correct

The full machine learning pipeline

Every ML project follows the same 5-step flow. This code covers all of them:

Step 1 — Load data

`datasets.load_iris()` → gives us X (features) and y (labels)

Step 2 — Preprocess

`y = (y == 0).astype(int)` → convert 3-class to binary. Real projects include cleaning, scaling, encoding.

Step 3 — Split

`train_test_split()` → 70% train, 30% test. Never let the model see test data during training.

Step 4 — Train

`svm.fit(X_train, y_train)` → model studies patterns. `svm.predict(X_test)` → applies what it learned.

Step 5 — Evaluate

`accuracy_score(y_test, y_pred)` → compare predictions to real answers. A score of ~0.98 = excellent.

Quick cheat sheet

Linear regression

Linear regression finds the best-fit straight line through data to predict a continuous output. It models the relationship between an independent variable X and dependent variable Y.

$$Y = \beta_0 + \beta_1 X$$

Where β_0 = intercept (constant), β_1 = slope, X = input feature, Y = predicted output.

The algorithm's goal is to find β_0 and β_1 values that minimise the prediction error.

R^2 (R-squared / Coefficient of Determination)

Measures how well predictions match actual values. R^2=1 means perfect prediction, R^2=0 means no better than the mean.

RMSE (Root Mean Squared Error)

The square root of average squared differences between predictions and actual values. Lower is better.

$$\varepsilon_i = y_{predicted} - y_i$$

Get Plus ×

R^2 Score (Coefficient of Determination)

Definition:

R^2 measures how well the regression model explains the **variance (spread)** in the data.

Formula

```
R² = 1 – ( Σ(yi – ŷi)² / Σ(yi – ȳ)² )
```

Where:

- `yi` = actual value
- `ŷi` = predicted value
- `ȳ` = mean of actual values

Meaning of Formula

- Numerator → error of model
- Denominator → total variation in data

👉 So:

R^2 = 1 – (error / total variation)

Mean Squared Error (MSE)

Definition:

Mean Squared Error (MSE) is a metric used in regression to measure the **average squared difference between actual values and predicted values.**

Formula

```
MSE = (1/n) Σ (yi – ŷi)²
```

Where:

- `yi` = actual value
- `ŷi` = predicted value
- `n` = number of data points

🔸 Key Idea

👉 It tells **how far predictions are from actual values**

👉 Squaring:

- removes negative signs
- penalizes large errors more

🔸 Interpretation

- **MSE = 0** → perfect prediction ✔
- **Higher MSE** → worse model ❌

🔹 Key Difference (Exam Point)

Feature	SVM	Linear Regression
Task	Classification	Regression
Output	0 or 1	Continuous value
Model	SVC	LinearRegression
Metric	Accuracy	MSE

🔥 One-line memory trick

👉 **SVM → classify (YES/NO)**

👉 **Linear Regression → predict number**

🔥 Final Exam Answer (Short Version)

👉 *Mean Squared Error (MSE) measures the average squared difference between actual and predicted values and is minimized during training of linear regression.*

👉 *R^2 score measures how well the model explains the variance in the data and is given by:*

```
R² = 1 – ( Σ(yi – ŷi)² / Σ(yi – ȳ)² )
```

👉 *MSE evaluates error, while R^2 evaluates overall model fit.*

🔹 Memory Trick

👉 **MSE → error size**
👉 **R^2 → model quality**

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2.$$

Calculating Testing Accuracy
Random Error (Residuals)
In regression, the difference between the observed value of the dependent variable(yi) and the predicted value(predicted) is called the residuals.

εi = ypredicted – yi

where ypredicted = B0 + B1 Xi

```
from sklearn.metrics import r2_score
print('====================LR Testing Accuracy================')
teachLR = r2_score(y_test, PredictionLR)
testingAccLR = teachLR * 100
print(testingAccLR)
```

Difference Between Classification and Regression

Definition

- **Classification:**
  A supervised learning task that predicts **discrete/categorical labels** (classes).
- **Regression:**
  A supervised learning task that predicts **continuous numerical values**.

Key Differences

Feature	Classification	Regression
Output type	Discrete (0,1,2...)	Continuous (real numbers)
Goal	Assign to a class	Predict a value
Example output	Yes/No, Spam/Not Spam	Price, Temperature
Models	SVM, Logistic Regression, KNN	Linear Regression, Ridge
Evaluation metrics	Accuracy, Precision, Recall	MSE, RMSE, R^2
Decision boundary	Yes (separates classes)	No (fits a line/curve)

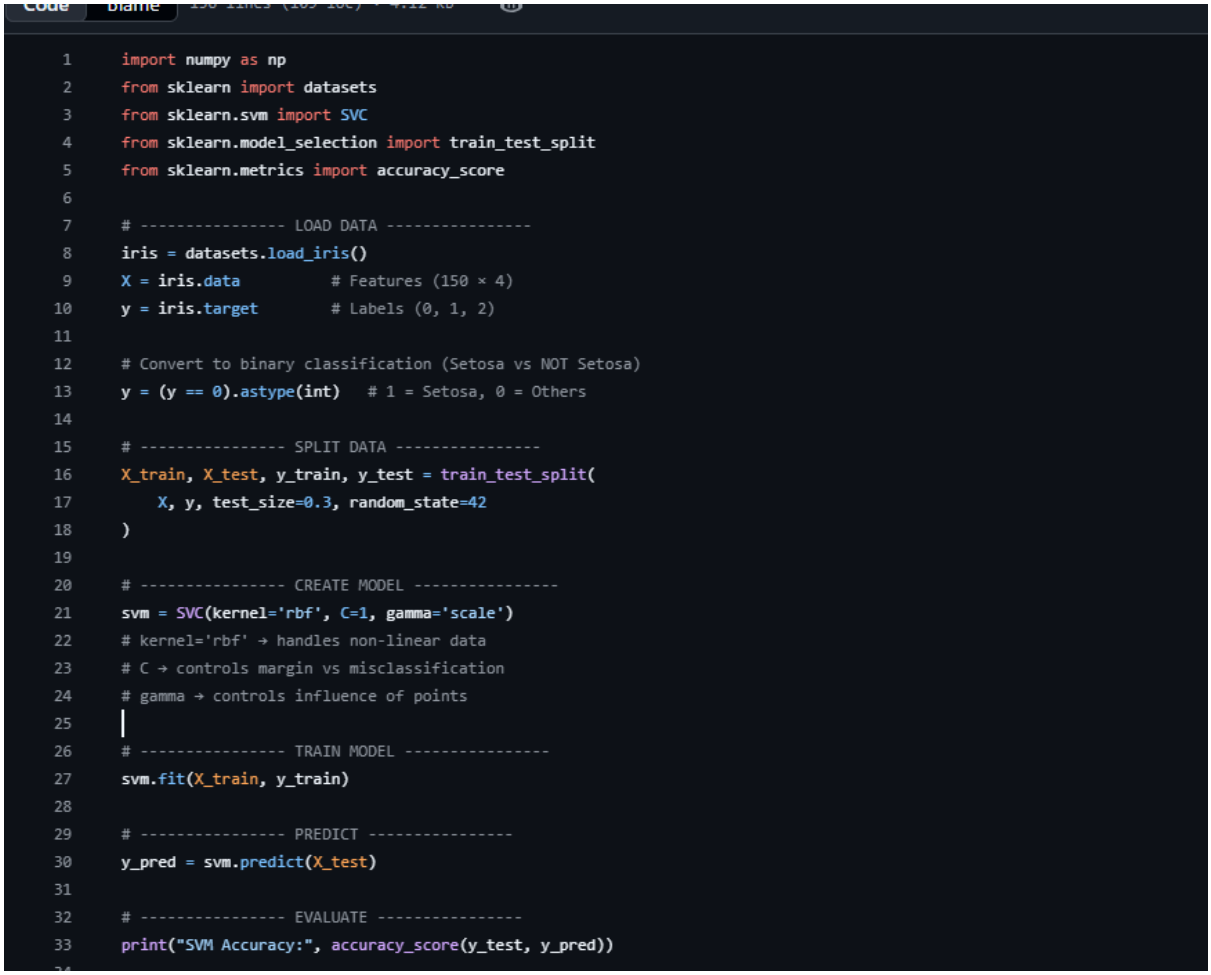
👉 *Classification predicts categorical class labels, while regression predicts continuous numerical values. Classification models are evaluated using accuracy-based metrics, whereas regression models use error-based metrics such as MSE and R^2.*

Quick Comparison (Now Easy to Revise)

Model	Task	Evaluation
SVM	Classification	Accuracy
Decision Tree	Classification	Accuracy
Linear Regression	Regression	MSE, R^2

🔥 Memory Trick

👉 **Tree & SVM → Accuracy**
👉 **Linear → Error (MSE, R^2)**



```python
import numpy as np
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# ---------------- LOAD DATA ----------------
iris = datasets.load_iris()
X = iris.data          # Features (150 × 4)
y = iris.target        # Labels (0, 1, 2)

# Convert to binary classification (Setosa vs NOT Setosa)
y = (y == 0).astype(int)   # 1 = Setosa, 0 = Others

# ---------------- SPLIT DATA ----------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# ---------------- CREATE MODEL ----------------
svm = SVC(kernel='rbf', C=1, gamma='scale')
# kernel='rbf' → handles non-linear data
# C → controls margin vs misclassification
# gamma → controls influence of points

# ---------------- TRAIN MODEL ----------------
svm.fit(X_train, y_train)

# ---------------- PREDICT ----------------
y_pred = svm.predict(X_test)

# ---------------- EVALUATE ----------------
print("SVM Accuracy:", accuracy_score(y_test, y_pred))
```

DECISION TREE

```python

import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# ---------------- LOAD DATA ----------------
# Example dummy data (replace with your dataset)
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([0, 1, 0, 1, 0])   # Binary classes

# ---------------- SPLIT DATA ----------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ---------------- CREATE MODEL ----------------
dt = DecisionTreeClassifier()

# ---------------- TRAIN MODEL ----------------
dt.fit(X_train, y_train)

# ---------------- PREDICT ----------------
y_pred = dt.predict(X_test)

# ---------------- EVALUATE ----------------

# Predictions
print("Predictions:", y_pred)

# Training Accuracy
train_acc = dt.score(X_train, y_train) * 100
print("DT Training Accuracy (%):", train_acc)

# Testing Accuracy
test_acc = accuracy_score(y_test, y_pred) * 100
print("DT Testing Accuracy (%):", test_acc)
```

REGRESSION’

```python
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# ---------------- LOAD DATA ----------------
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 5, 4, 5])

# ---------------- SPLIT DATA ----------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ---------------- CREATE MODEL ----------------
lr = LinearRegression()

# ---------------- TRAIN MODEL ----------------
lr.fit(X_train, y_train)

# ---------------- PREDICT ----------------
y_pred = lr.predict(X_test)

# ---------------- EVALUATE ----------------

# 1. Mean Squared Error (actual regression metric)
mse = mean_squared_error(y_test, y_pred)

# 2. R² Score
r2 = r2_score(y_test, y_pred)

# 3. "Accuracy" style (R² in percentage form)
accuracy = r2 * 100

print("Predictions:", y_pred)
print("MSE:", mse)
print("R2 Score:", r2)
print("LR Testing Accuracy (%):", accuracy)

```

PANDAS

🔹 1. What is `df`?

👉 `df` stands for **DataFrame**

It comes from:

```python
import pandas as pd
df = pd.read_csv("file.csv")
```

So:
👉 `df` = **table of data (like Excel sheet in Python)**

🔥 Think of it like this:

spending	age	visits	label
5000	25	10	1
2000	40	3	0

👉 This whole table = **df (DataFrame)**

🔹 2. Pandas vs NumPy (VERY IMPORTANT)

Feature	Pandas	NumPy
Data type	Table (rows + columns)	Arrays
Labels	Yes (column names)	No
Use	Real-world datasets	Math operations

👉 In ML:

- **Pandas → data handling**
- **NumPy → calculations**

3. Most important df opera U MUST KNOW)

Load data

```python
df = pd.read_csv("file.csv")
```

👉 Reads file into table

See data

```python
df.head()     # first 5 rows
df.info()     # structure
df.shape      # (rows, columns)
```

Select columns

```python
X = df.drop("label", axis=1)   # all columns except label
y = df["label"]                # only label column
```

👉 VERY COMMON in ML

Handle missing values

```python
df.fillna(df.mean(numeric_only=True), inplace=True)
```

👉 Fill missing numbers with average

Filter rows (remove outliers)

```python
df = df[df["spending"] < 10000]
```

👉 Keep only valid rows

Convert to NumPy (optional)

```python
X = df.values
```

🔹 4. How df connects to ML models

Models like SVM expect:

```
X → features (input)
y → labels (output)
```

From df:

Python Run

```python
X = df.drop("label", axis=1)
y = df["label"]
```

👉 That's it.

🔥 5. Your full task rewritten S

Get Plus ×

Let's simplify your customer classification task:

Step 1: Load data

Python Run

```python
df = pd.read_csv("customers.csv")
```

Step 2: Clean data

Python Run

```python
df.fillna(df.mean(numeric_only=True), inplace=True)
```

Step 3: Split features and label

Python Run

```python
X = df.drop("label", axis=1)
y = df["label"]
```

🔥 1. `dropna()` = remove missing data

Python Run

```python
df.dropna(inplace=True)
```

👉 This means:

- Remove any row that has missing values

🔥 4. Why `df["text"]` is used in spam example?

Get Plus ×

Because dataset looks like:

text	label
email content	1

👉 We ONLY want:

- text → features
- label → target

So it is clearer to write:

```python
X = df["text"]
```

Python Run

🔹 5. Why NOT always use `drop()`?

Because:

❌ Problem with drop:

- Works best when many feature columns exist
- Slightly more complex
- Not needed if only one column is required

🔥 7. Why your confusion happens

Because:

Situation	Best method
Single feature column	`df["column"]`
Many feature columns	`df.drop()`

🔥 2. `fillna()` = replace missing data

Python

```python
df.fillna(df.mean(), inplace=True)
```

👉 This means:

- Replace missing values instead of removing them

✔ When we use it

We use `fillna()` when:

- Dataset is small (we don't want to lose data)
- Missing values are not too many
- We want to **keep all rows**

🔥 4. When NOT to use `dropna()`

❌ Don't use dropna when:

- Dataset is small
- Too many missing values
- Losing data hurts model learning

🔥 5. When NOT to use `fillna()`

❌ Don't use fillna when:

- Missing data is random but large
- Filling would introduce fake patterns

Step 4: Train model

Python

```python
model.fit(X, y)
```

👉 Notice:

df is only used BEFORE model

Get Plus ×

🔹 6. Why pandas is used (IMPORTANT)

👉 Real datasets:

- messy
- missing values
- text + numbers

👉 Pandas helps:

✔ clean data
✔ organize data
✔ prepare for ML

🔥 7. One-line intuition

👉 **df = Excel sheet in Python**

🔹 8. Minimal cheat sheet (memorize this)

Python Run

```python
df = pd.read_csv("file.csv")

X = df.drop("label", axis=1)
y = df["label"]

df.fillna(df.mean(numeric_only=True), inplace=True)
```

🔥 9. Biggest mistake beginners make

❌ Thinking df is complex
✔ It's just a **table**

🔹 Final understanding

- Pandas (df) → handles data
- NumPy → handles math
- Sklearn → builds models

🔥 Short answer first

👉 ❌ `TfidfVectorizer` is NOT the same as `get_dummies()`

👉 ✔ Both convert data into numbers, but for **completely different types of data**

🔹 1. `get_dummies()` = categorical encoding

Used for:

👉 Structured data (tables)

Example:

city
Karachi
Lahore
Karachi

After `get_dummies()`:

✦ Get Plus ×

```
Karachi | Lahore
1       | 0
0       | 1
1       | 0
```

👉 Converts **categories → binary columns**

🔹 2. `TfidfVectorizer` = text understanding

Used for:

👉 Unstructured data (text)

Example:

```
"I love machine learning"
"machine learning is fun"
```

After TF-IDF:

Get Plus ×

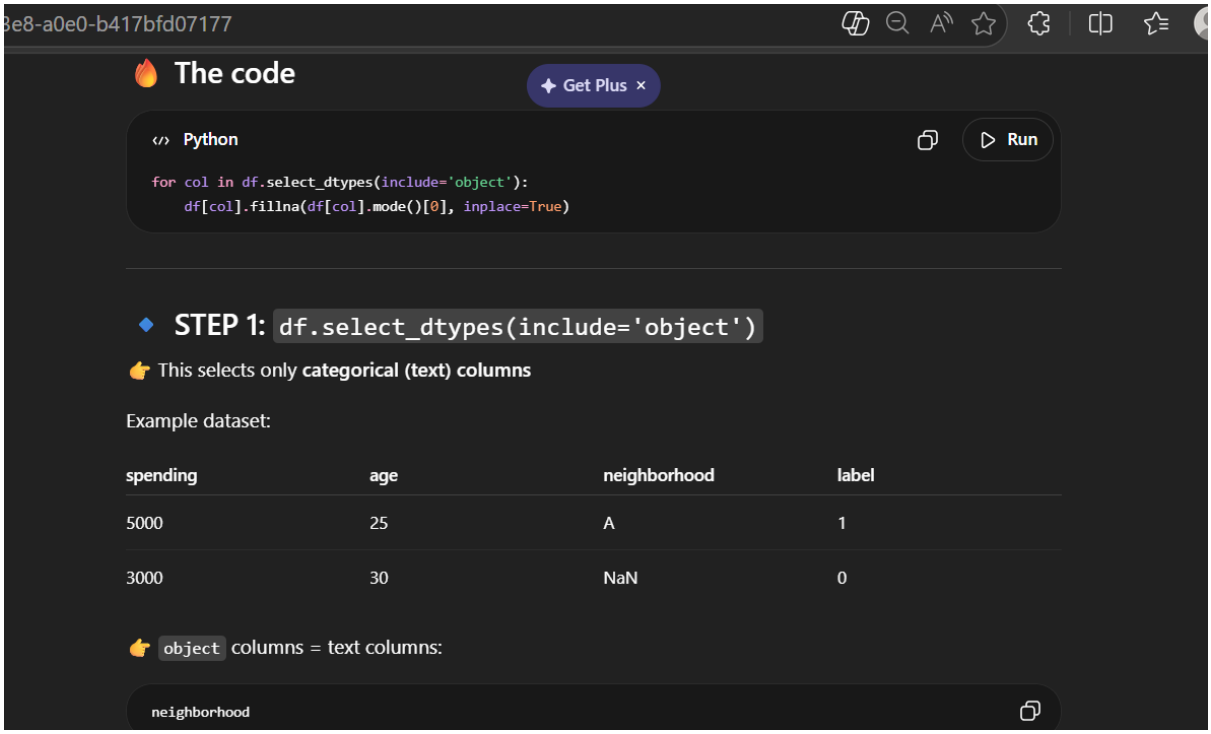
```
word → importance score

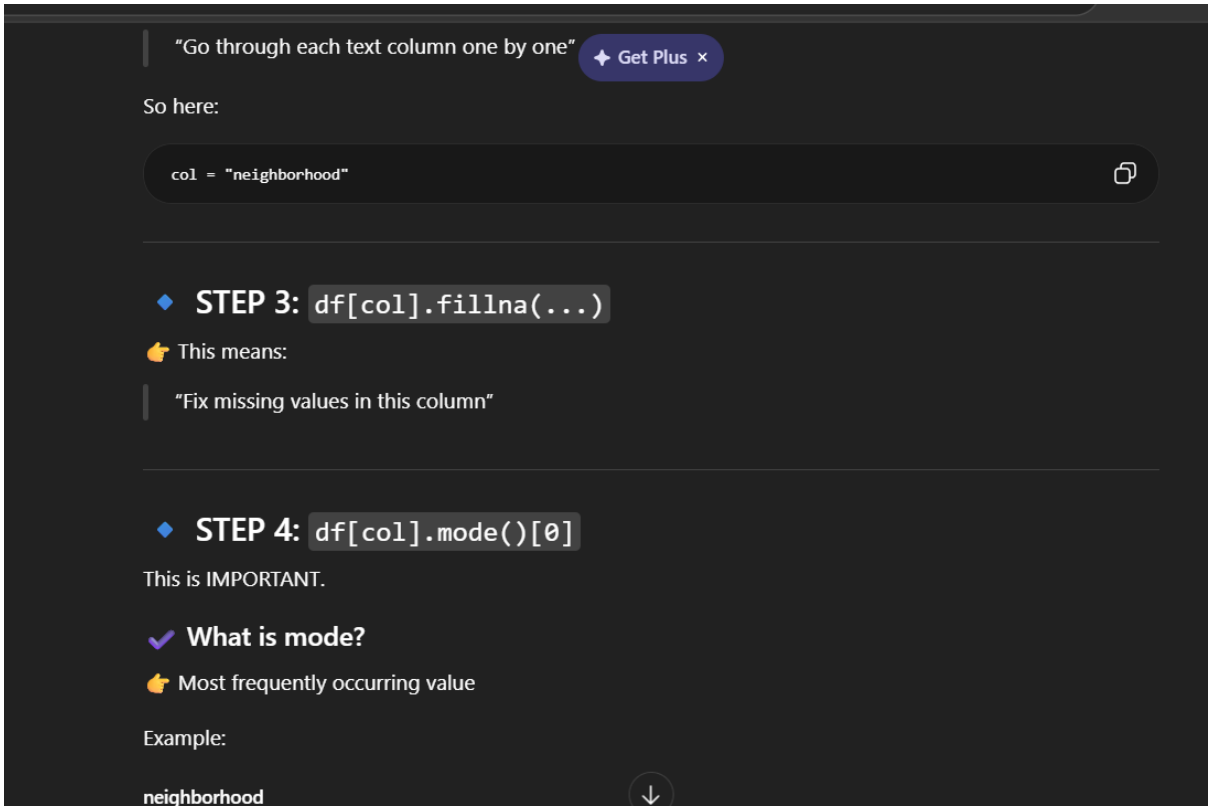
love      → 0.7
machine   → 0.5
learning  → 0.6
fun       → 0.4
```

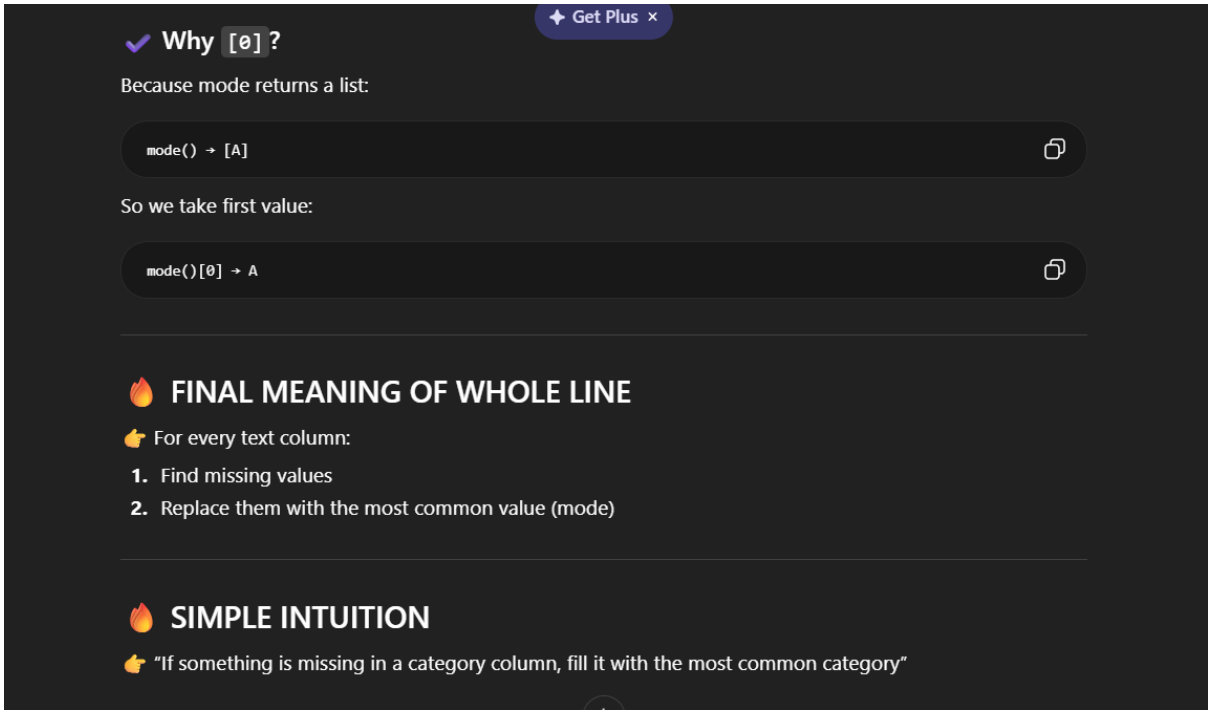
👉 Converts **words → importance numbers**

🔥 3. Key difference (VERY IMPORTANT)

Feature	get_dummies
Input type	Categorical (structured)
Output	Binary columns (0/1)
Meaning	presence/absence
Used in	tables (neighborhood, city)


3e8-a0e0-b417bfd07177
The code
Get Plus ×
Python
Run
for col in df.select_dtypes(include='object'):
df[col].fillna(df[col].mode()[0], inplace=True)
STEP 1: df.select_dtypes(include='object')
👉 This selects only categorical (text) columns
Example dataset:
spending age neighborhood label
5000 25 A 1
3000 30 NaN 0
👉 object columns = text columns:
neighborhood


"Go through each text column one by one"
Get Plus ×
So here:
col = "neighborhood"
STEP 3: df[col].fillna(...)
👉 This means:
"Fix missing values in this column"
STEP 4: df[col].mode()[0]
This is IMPORTANT.
What is mode?
👉 Most frequently occurring value
Example:
neighborhood



TASK # 3

You work for a retail business and your task is to classify customers into two categories: high-value and low-value customers. The classification is based on customer features such as total spending in the last 6 months, age, number of visits, and purchase frequency. You have a dataset with customer information, including spending habits, frequency of visits, and demographics. The goal is to classify customers into high-value (1) and low-value (0) categories.

- Clean the dataset and handle any missing or outlier values. Perform feature scaling if necessary.
- Divide the dataset into a training set and a testing set.
- Find a separating hyperplane for classification.
- Find rules that classify customers based on their features.
- Evaluate the performance of the model.

This is about **evaluation choice.**

🔥 Rule to remember (VERY IMPORTANT)

Problem Type	Metric
Classification	accuracy, classification_report
Regression	MSE, R^2

🔹 Why classification_report?

Because it gives:

```
precision
recall
f1-score
```

🔹 1. "How do you write rules here?"

You're referring to:

```python
from sklearn.tree import export_text
rules = export_text(dt, feature_names=list(X.columns))
print(rules)
```

🔥 What this actually does

👉 It **automatically generates rules** from the trained Decision Tree.

You are **NOT writing rules manually** ❌
👉 The model learns them from data ✔

- Predict the price of a house given a new set of features.

TASK # 2

You work as a data scientist for an email service provider. Your task is to develop a model that can classify emails as spam or not spam based on their content. You have a labeled dataset with thousands of emails. Each email has features such as frequency of specific words, length of the email, presence of hyperlinks, and sender's address. The task is to classify an email as spam (1) or not spam (0).

- Preprocess the dataset by converting text features to numerical features.
- Train a model to classify the emails based on their features.
- Evaluate the model's performance.
- Deploy the model to classify new incoming emails.

TASK # 3

Great — this is a **classic spam classification lab task**. I'll give you a **complete, clean solution** in the SAME format you've been using so you can revise easily.

We'll use:
👉 **TF-IDF** for text → numbers
👉 **SVM (or you can switch to Naive Bayes easily)**

K-Means Clustering — Background

K-Means is one of the simplest and most popular **unsupervised machine learning algorithms**.

Before understanding K-Means, first understand:

What is Unsupervised Learning?

In supervised learning, we have:

- **Input data (X)**
- **Correct output labels (Y)**

Example:

- Study hours → Grade

But in **unsupervised learning**, we only have data, no labels.

Example customer data:

Customer	Income	Spending Score
A	15k	39
B	16k	81
C	70k	6

Example customer data:

Customer	Income	Spending Score
A	15k	39
B	16k	81
C	70k	6

No one tells us:

- rich spender
- rich saver
- low-income spender

The algorithm must discover patterns itself.

That's where **clustering** comes in.

Get Plus ×

What is Clustering?

Clustering means:

> Grouping similar data points together.

Example:

Suppose a mall has customer data:

- Age
- Income
- Spending score

K-means can automatically group customers into:

Cluster 1

- Low income
- Low spending

Cluster 2

- High income
- High spending

Cluster 3

- High income
- Low spending

This helps businesses target customers better.

Purpose of K-Means

K-Means is used to:

1. Customer Segmentation

Group customers by buying behavior.

Example:

- Premium customers
- Budget customers
- Frequent buyers

---

2. Image Compression

Reduce number of colors in images.

---

3. Document Clustering

Why is it called K-Means?

Two parts:

K

Number of clusters.

Example:

- K = 3 → 3 groups

Means

Mean = average.

Each cluster has a center point called:

Centroid

Centroid = average of all points in that cluster.

Get Plus ×

Means

Mean = average.

Each cluster has a center point called:

Centroid

Centroid = average of all points in that cluster.

Example:

Cluster points:
(2,3), (4,5), (6,7)

Centroid:

$$x = (2+4+6)/3 = 4$$

$$y = (3+5+7)/3 = 5$$

Centroid = (4,5)

How K-Means Algorithm Work

Get Plus ×

Suppose we have points:

Python Run

and:

Python Run

```
K = 2
```

Means 2 clusters.

Step 1: Choose K

Choose number of clusters.

Example:

Python Run

```
K = 2
```

We want 2 groups.

Step 2: Initialize Random Centroids

Randomly choose 2 centroids.

Example:

Python

Step 3: Assign Each Point to Closest Centroid

Distance formula:

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

Assign points to nearest centroid.

Example:

Point (2,1)

- closer to C1

Point (9,8)

- closer to C2

Now clusters become:

Cluster 1:

Python Run

Cluster 2:

Python Run

```
(8,9), (9,8)
```

Step 4: Recalculate New Centroids

Take average.

Cluster 1:

$$((1+2)/2, (2+1)/2) = (1.5, 1.5)$$

Cluster 2:

$$((8+9)/2, (9+8)/2) = (8.5, 8.5)$$

New centroids:

- C1 = (1.5,1.5)
- C2 = (8.5,8.5)

1. Centroid

Center of cluster.

---

2. Cluster

Group of similar points.

---

3. Distance

Usually Euclidean distance.

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

---

4. Inertia / WCSS

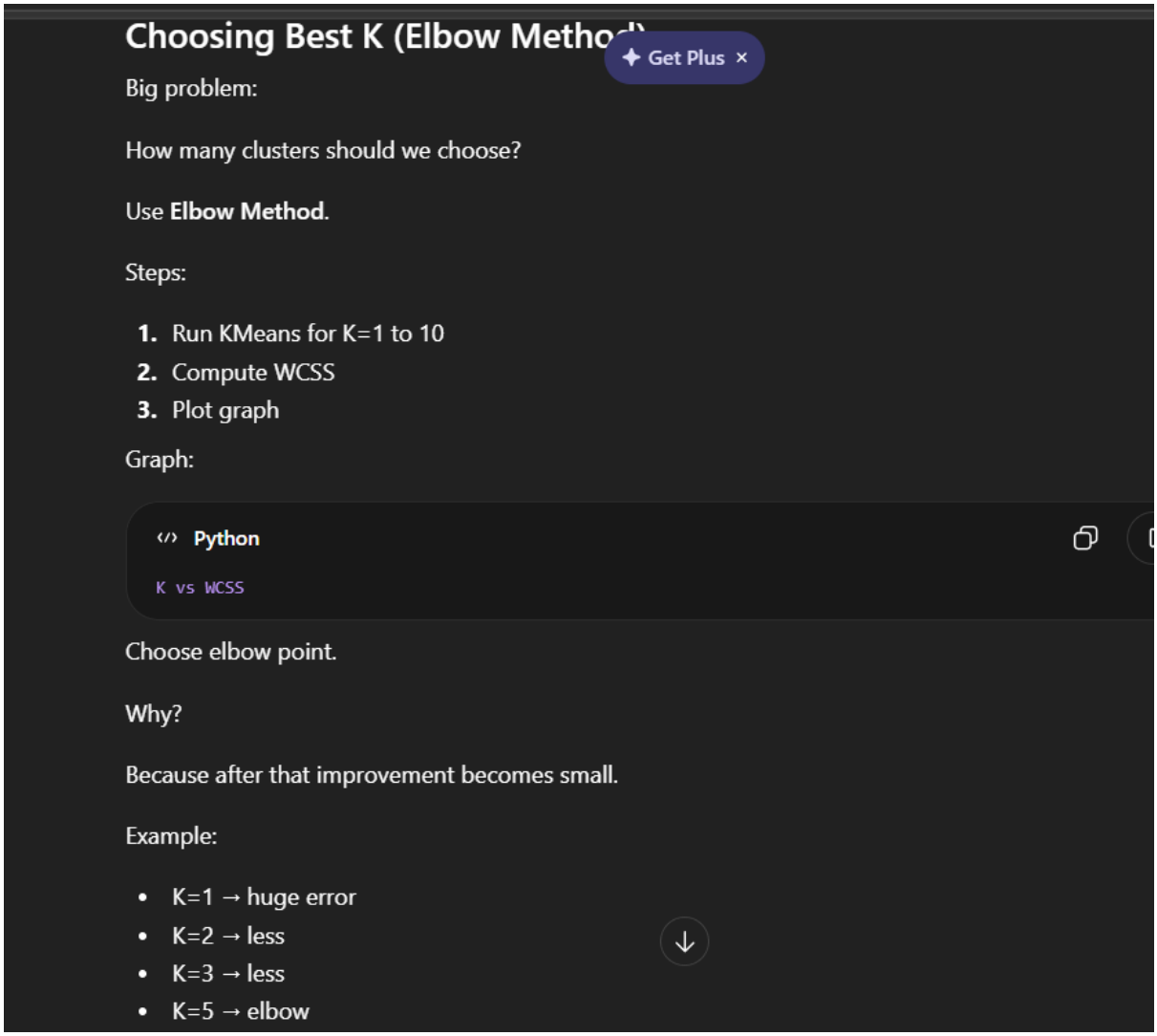
WCSS = Within Cluster Sum of Squares

Measures compactness.

Formula idea:

$$\sum (point - centroid)^2$$

Lower WCSS = better clustering.


Choosing Best K (Elbow Metho
Get Plus ×
Big problem:
How many clusters should we choose?
Use Elbow Method.
Steps:
1. Run KMeans for K=1 to 10
2. Compute WCSS
3. Plot graph
Graph:
Python
K vs WCSS
Choose elbow point.
Why?
Because after that improvement becomes small.
Example:
• K=1 → huge error
• K=2 → less
• K=3 → less
• K=5 → elbow

KMEANS

🧠 SUPER SIMPLE UNDERSTA[Get Plus ×]EMORIZE THIS FLOW)

Think of it like a **7-step story**:

🔹 Step 1: Import

👉 Bring tools

```python
import numpy, pandas, matplotlib, sklearn
```

🔹 Step 2: Load Data

👉 Read file

```python
df = pd.read_csv()
```

🔹 Step 3: Pick Important Columns

[Get Plus ×]

👉 Only take useful data

```python
X = df.iloc[:, [3,4]]
```

(Income + Spending)

🔹 Step 4: Find Best K (Elbow)

👉 Try K = 1 to 10

```python
for i in range(1,11):
```

Store error:

```python
wcss.append(kmeans.inertia_)
```

Plot graph → find elbow

🔹 Step 5: Scale Data

👉 Make values balanced

Python Run

```
StandardScaler()
```

🔹 Step 6: Train Model

👉 Apply K-Means

Python Run

```
kmeans = KMeans(n_clusters=5)
y_pred = kmeans.fit_predict()
```

🔹 Step 7: Plot Clusters

👉 Show groups

Python Run

```
plt.scatter(...)
```

🔷 First understand this variable

Python Run

```
y_pred
```

It looks like:

Python Run

```
[0, 1, 2, 0, 4, 3, 1, ...]
```

👉 Meaning:

- Data point 1 → cluster 0
- Data point 2 → cluster 1
- Data point 3 → cluster 2
- etc.

🔹 Step 1: `y_pred == 0`

This creates a **filter (True/False)**:

Example:

```python
y_pred = [0,1,0,2]
```

Then:

```python
y_pred == 0 → [True, False, True, False]
```

👉 Means:
"Select only points in **cluster 0**"

🔹 Step 2: `X[y_pred == 0]`

This gives:

👉 Only those rows that belong to cluster 0

🔹 Step 3: `X[y_pred == 0, 0]`

Now:

- `,0` → take **column 0** (Income)
- `,1` → take **column 1** (Spending Score)

🔷 So this line means:

```python
X[y_pred == 0, 0]
```

Run

👉 Income of cluster 0 points

```python
X[y_pred == 0, 1]
```

Run

👉 Spending score of cluster 0 points

🔷 Now the full line

Python

```
plt.scatter(X[y_pred == 0, 0],
            X[y_pred == 0, 1],
            c='blue',
            label='Cluster 1')
```

Means:

👉 Plot all points of **cluster 0**

- X-axis → Income
- Y-axis → Spending Score
- Color → Blue

🔷 Same logic for all clusters

Python

```
y_pred == 1 → Cluster 2 (green)
y_pred == 2 → Cluster 3 (red)
y_pred == 3 → Cluster 4 (black)
y_pred == 4 → Cluster 5 (purple)
```

Each line just:
👉 picks one cluster
👉 plots it with a different color

🔷 Now Centroids (VERY IMPORTANT)

Python

```
kmeans.cluster_centers_
```

This stores:

👉 coordinates of all cluster centers

Example:

Python

```
[[40, 60],
 [70, 20],
 [25, 80],
 ...]
```

Python Run

```
kmeans.cluster_centers_[:, 1]
```

👉 All Y values (Spending Score)

Final centroid plot:

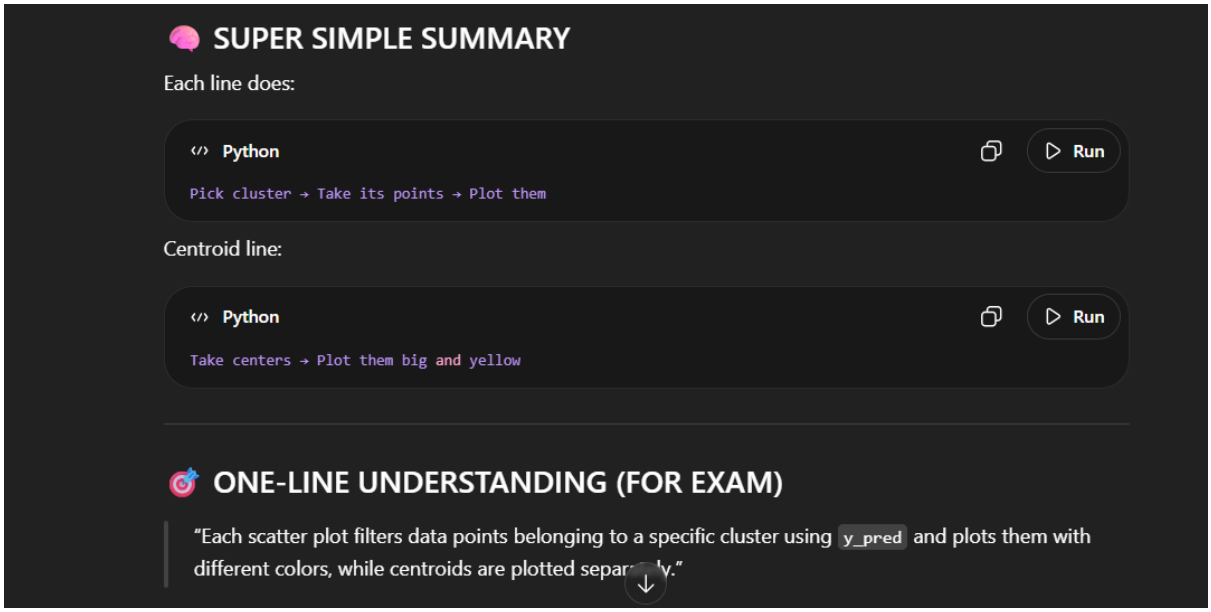
Python Run

```
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            c='yellow', s=300, label='Centroids')
```

Means:

👉 Plot centroids as:

- Yellow color
- Big size (`s=300`)



MM & HMM

🔷 1. What is a Markov Model?

A **Markov model** is a way to model systems where:

> The future depends only on the present, not the past.

This is called the **Markov Property**.

💡 Simple idea:

If you're in a state right now, the next state depends ONLY on where you are now.

Not:

- how you got there ❌
- how long you stayed there ❌
- previous history ❌

Only:

- current state ✔

🔷 2. Real-life intuition

Think of weather:

States:

- Sunny 🌞
- Rainy 🌧️

Example rule:

- If today is Sunny → tomorrow has 70% Sunny, 30% Rainy
- If today is Rainy → tomorrow has 40% Sunny, 60% Rainy

So:
👉 Tomorrow depends only on today

That's a Markov model.

🔷 3. Key Components of a Markov Model

1. States

All possible conditions:

Python Run

```python
states = ["Red", "Blue"]
```

So system can be in only:

- Red
- Blue

2. Transition Probability Matrix

This is the MOST important part:

```python
transition_matrix = np.array([
    [0.5, 0.5],  # From Red
    [0.5, 0.5]   # From Blue
])
```

Interpretation:

From \ To	Red	Blue
Red	0.5	0.5
Blue	0.5	0.5

Meaning:

If you're in Red:

- 50% chance stay Red
- 50% chance go Blue

- 50% chance stay Red
- 50% chance go Blue

If you're in Blue:

- 50% Red
- 50% Blue

👉 This matrix fully defines the system.

🔷 4. What is happening in your code?

Let's break it step by step.

Function start

```python
def simulate_markov_process(initial_state, num_steps):
```

You are simulating how the system moves step by step.

Starting point

```python
current_state = initial_state
state_sequence = [current_state]
```

If you start from:

```python
"Red"
```

Then:

- current_state = Red
- sequence = ["Red"]

Main loop (the simulation engine)

```python
for _ in range(num_steps):
```

This repeats the process step-by-step (like time steps).

Decision based on current state

If currently Red:

```python
if current_state == "Red":
    next_state = np.random.choice(states,
                    p=transition_matrix[0])
```

🔹 Decision based on current state

If currently Red:

```python
if current_state == "Red":
    next_state = np.random.choice(states,
                    p=transition_matrix[0])
```

Here:

```python
transition_matrix[0] = [0.5, 0.5]
```

So:

- 50% Red
- 50% Blue

👉 Python randomly chooses next state based on probabilities

If currently Blue:

```python
else:
    next_state = np.random.choice(states,
                    p=transition_matrix[1])
```

Same logic:

- 50% Red
- 50% Blue

🔹 Update system state

```python
state_sequence.append(next_state)
current_state = next_state
```

Now:

- save new state
- move forward in time

🔹 Return full history

```python
return state_sequence
```

So you get something like:

```
Red → Blue → Blue → Red → Blue → ...
```

🔷 5. What your simulation is doing conceptually

You are simulating a **random walk on states**:

Each step:

1. Look at current state
2. Use probability rules (matrix)
3. Randomly choose next state
4. Move forward

HMM

🧠 What is HMM (Hidden Markov Model)?

An **HMM** is a statistical model used when:

> ❗ You can see the output (observations), but NOT the actual system state (hidden state).

🔹 Simple Idea

You see:

- Umbrella ☔ / No umbrella 🌤️

But you **don't directly see the weather**

Hidden states:

- Sunny ☀️
- Cloudy ☁️
- Rainy 🌧️

So:

> Weather is hidden
> Umbrella is observed

🎯 Why "Hidden"?

Because:

- We observe **outputs**
- We guess the **hidden cause**

Example:

Hidden State (Weather)	Observation (What we see)
Sunny	No umbrella
Rainy	Umbrella

🔷 3 MAIN PARTS OF HMM

1. Initial Probability (π)

👉 Where do we start?

Example:

- Sunny = 0.6
- Cloudy = 0.3
- Rainy = 0.1

- Rainy = 0.1

---

2. Transition Matrix (A)

👉 How weather changes over time

Example:

- Sunny → Rainy (0.1)
- Rainy → Cloudy (0.3)

So:

> "From current state → next state probability"

---

3. Emission Matrix (B)

👉 What we observe from a state

Example:

- Rainy → Umbrella (0.9)
- Sunny → No Umbrella (0.9)

So:

> "Hidden state → visible output"

🧠 FULL FLOW OF HMM

```
Hidden State → Transition → New Hidden State
      ↓
   Emit Observation
      ↓
   We observe output
```

Python Get Plus × Run

```python
import numpy as np
```

Used for probabilities and random choices.

🔹 Step 2: Define states

Python Run

```python
states = ["Sunny", "Cloudy", "Rainy"]
observations = ["Umbrella", "No Umbrella"]
```

Hidden vs visible world.

🔹 Step 3: Initial probabilities (π)

Python Run

```python
pi = np.array([0.6, 0.3, 0.1])
```

Start chance:

- Sunny most likely

🔹 Step 4: Transition matrix (A)

Python Run

```python
A = [
 [0.7, 0.2, 0.1],
 [0.3, 0.4, 0.3],
 [0.2, 0.3, 0.5]
]
```

Meaning:

From Sunny:

- stay Sunny = 0.7
- go Cloudy = 0.2
- go Rainy = 0.1

Step 5: Emission matrix (B)

Python Run

```
B = [
 [0.1, 0.9],
 [0.4, 0.6],
 [0.9, 0.1]
]
```

Meaning:

Sunny:

- Umbrella = 0.1
- No umbrella = 0.9

Rainy:

- Umbrella = 0.9

Step 7: Pick starting state

Python Run

```
state_idx = np.random.choice(len(states), p=pi)
```

Random start based on π.

Step 8: Loop (time steps)

Python Run

```
for _ in range(num_steps):
```

Repeat process over time.

Step 9: Generate observation

Python Run

```
obs_idx = np.random.choice(len(observations), p=B[state_idx])
```

From hidden state → we see umbrella or not.

🔹 Step 10: Move to next state

Python

```
state_idx = np.random.choice(len(states), p=A[state_idx])
```

Weather changes over time.

🔹 Step 11: Store results

Python

```
hidden.append(states[state_idx])
observed.append(observations[obs_idx])
```

Save both:

- hidden weather
- observed umbrella

🔹 Step 12: Output

Python

```
print("Hidden:", hidden_states)
print("Observed:", obs_seq)
```